

Graphics Processing Units (Part 2)

Contents

- ❑ GPU definition
- ❑ Motivation and examples
- ❑ GPU History
- ❑ GPU companies
- ❑ Processing capability of GPU vs. CPU
- ❑ Graphics pipeline
- ❑ Graphics pipeline bottlenecks
- ❑ Haptics rendering pipeline
- ❑ Classic GPU pipeline
- ❑ GPU graphics pipeline
- ❑ Classic vs. unified shader architecture
- ❑ Unified shader architecture
- ❑ NVIDIA GeForce 8800
- ❑ Shader languages
- ❑ Software technologies
- ❑ Distributed visualization
- ❑ Sort-first pipeline
- ❑ Sort-middle pipeline
- ❑ Sort-last pipeline
- ❑ Load Balancing
- ❑ References

GPU History

1. Before 1998)

- Ex. NVIDIA RIVA TNT, ATI Rage, Matrox G400, and 3dfx Voodoo3.
- Triangle rasterization with textures.
- No vertex transformation.
- Limited set of math operations to combine textures for pixel's color.

2. 1999-2000

- NVIDIA GeForce 256 and GeForce2, ATI Radeon 7500, and S3 Savage3D
- Offload vertex transformations and lighting from the CPU to the GPU
- Increase the number of fixed pixel-pipelines
- Configurable, but not yet programmable

GPU History

1. 2001

- NVIDIA GeForce3 and GeForce4 Ti, Microsoft Xbox, and ATI Radeon 8500
- Vertex programmability (the first programmable GPU)
- Pixel level partially configuration

2. 2002 - 2006

- NVIDIA GeForce FX family (i.e. 6800, 7800), ATI Radeon 9700
- Vertex and fragment programmability
- NVIDIA 7950 and ATI Radeon X1950
- NVIDIA creates CUDA architecture

3. 2007

- Unified Shader Architecture (vertex, geometry, and fragment processors)
- GPGPU, NVIDIA GeForce 8800. Tesla is the first dedicated General Purpose GPU

GPU History

6. After 2006

- Unified Shader Architecture (vertex, geometry, and fragment processors)
- GPGPU, NVIDIA GeForce 8, 9, 200, and 400 Series (e.g. 8800), support DirectX 10, CUDA - first C programming language API for GPGPU
- Tegra mobile processor, Tesla based first supercomputer, Apple adopts GeForce 9400M for MacBook, PhysX – physics engine (2008)
- GeForce 400 Series: April 2010, Fermi Chip, 512 stream processors, in 16 groups of 32 processors, support OpenGL 4.0 and Direct3D 11.
- GeForce 400 Series (November 2010), GTX 580 (20% faster than GTX 480)
- Kepler architecture, GeForce 600, Tegra 3 on tablets, virtualized GPU onto Cloud (2012)
- Tegra 4 quad-core mobile processor (2013)

GPU History

Last product:

- GeForce GTX TITAN Z is a dual GPU gaming engine, 5760 cores and 12 GB of memory.

Memory interface 768 bits (384 bits/ GPU processor), Memory bandwidth 672 GB/sec.

Maximum digital resolution 4096x2160.

Tendency:

- Unify the GPU and CPU functionality within the same chip (e.g. join NVIDIA GPU and ARM RISC architecture)
- Graphics cluster of GPUs
- GPGPU cluster in heterogeneous architectures (e.g. Grid, Cloud)

GPU companies

- ❑ **AMD** (producer of the ATI Radeon graphics chip line)

AMD (Advanced Micro Devices) takes control over ATI in 2006 and produces the Radeon graphics chipsets under the AMD brand: AMD 580X CrossFire, AMD 690G, <http://www.amd.com/>

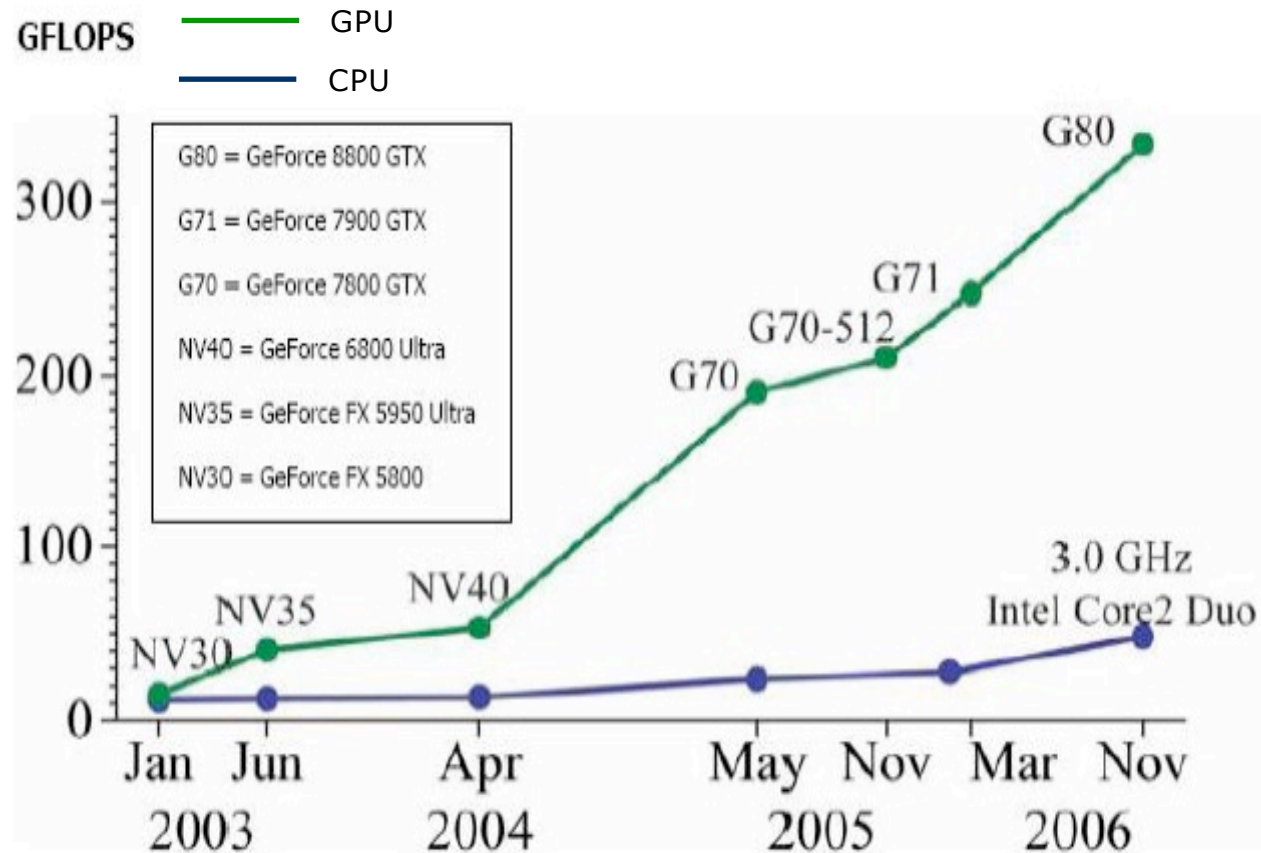
- ❑ **NVIDIA** (producer of the NVIDIA Geforce graphics chip line), founded in 1993. <http://www.nvidia.com/>

- ❑ **Intel** also produces GPUs that are built into their motherboards, such as the 915 and 945. These chips are less optimum for playing 3D games. <http://www.intel.com>

- ❑ **S3 Graphics** (purchased by HTC in 2011), founded in 1989. Produces ViRGE, Savage 3D, SavageXP, Chrome series, <http://www.s3graphics.com/>

- ❑ **Matrox**, founded in 1978, <http://www.matrox.com/>. Produces graphics card Matrox Mystique, Matrox G400, Matrox Parhelia.

Processing capability of GPU vs. CPU



GPU definition

- In 1999 NVIDIA invents the graphics processing unit, GeForce 256, defined as:
“a single-chip processor with integrated transform, lighting, triangle setup/clipping, and rendering engines that is capable of processing a minimum of 10 million polygons per second”.
The modern GPUs process over 2×10^9 polygons per second.

Performance GeForce 6, 7 and 8 series

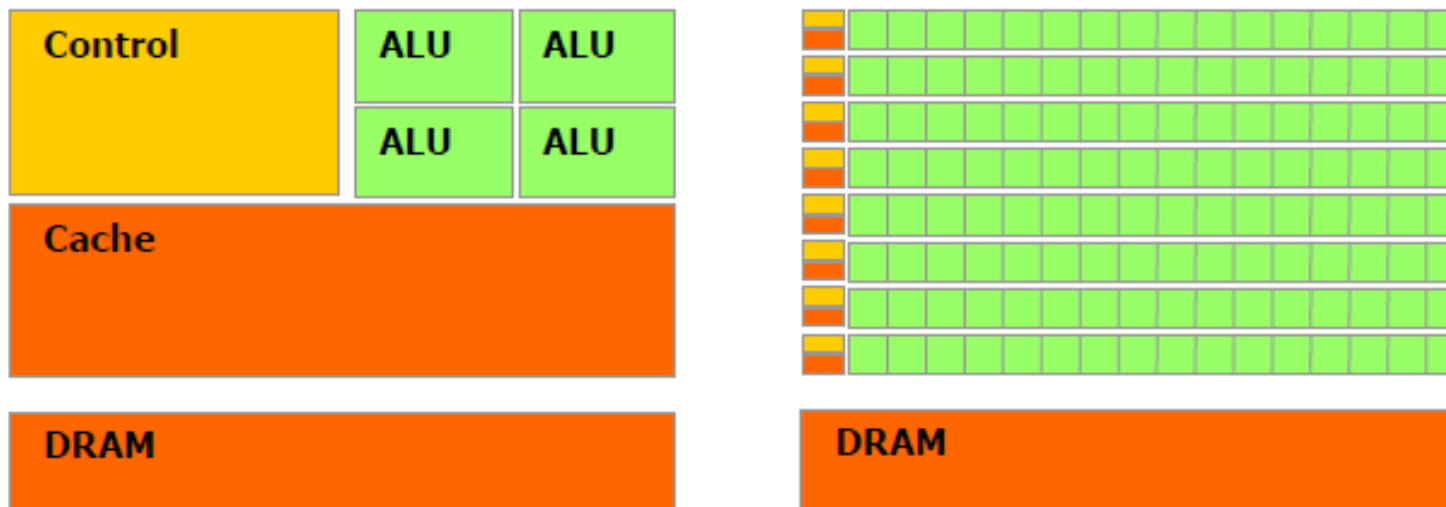
	Geforce 6800 Ultra	Geforce 7800	Geforce 8800 Ultra
Stream processors			128
Memory bandwidth	35.2 GB/sec	54.4 GB/sec	103.7 GB/sec
Fill rate (texels/sec.)	6.4 billion	13.2 billion	39.2 billion
Vertices per second	600 million	1100 million	
Pixel per clock (peak)	16	24	48
Performance	53 Gigaflops	200 Gigaflops	330 Gigaflops

GPU vs CPU

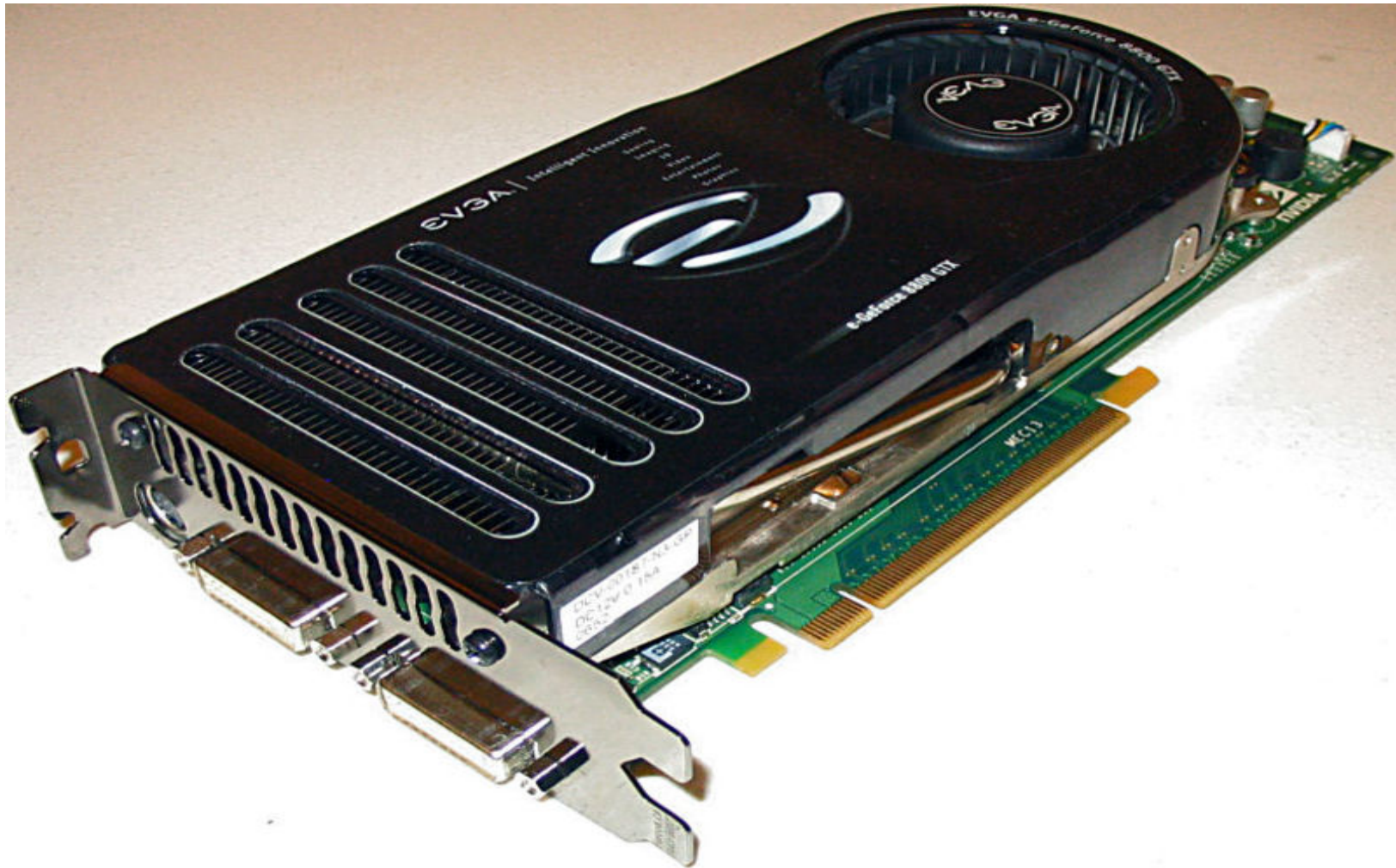
- ❑ GPU is designed for high parallel operations while CPU executes programs serially
- ❑ GPU has many parallel execution units and higher transistor counts, while CPU has a few execution units and higher clockspeeds
- ❑ GPU has much deeper pipelines (several thousand stages vs 10-20 for CPU)
- ❑ GPU has significant faster and more advanced memory interfaces as they need to shift around a lot more data than CPU

GPU vs CPU Architecture

- ❑ GPU is specialized for compute-intensive, highly parallel computation, and graphics rendering
- ❑ Designed such that more transistors are devoted to data processing rather than data caching and flow control



NVIDIA GeForce 8800 GTX

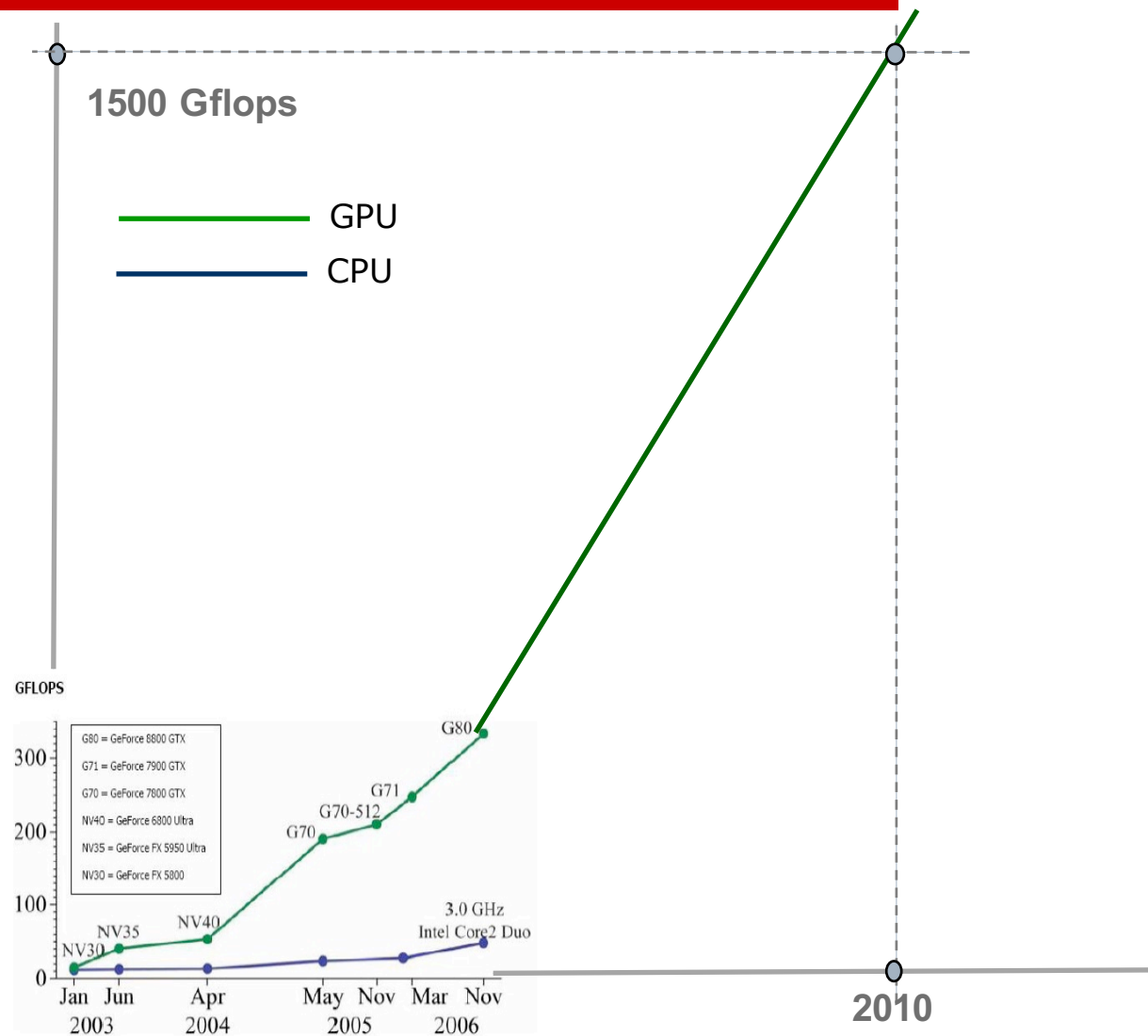


Picture taken from Wikipedia, the free encyclopedia.

Performance GeForce GTX performance

	Geforge 9800 GTX	GeForce GTX 285	GeForce GTX 480	GeForce GTX 580	GeForce GTX 590
Stream (Shader) processors	128	240	480	512	1024
Core Clock (MHz)	675	648	700	772	1215
Memory bandwidth	70.4 GB/s	159 GB/s	177 GB/s	192. GB/s	327.7 GB/s
Memory amount	512 MB	1024 MB	1536 MB	1536 MB	3072 MB
Memory bus width	256 bits	512 bits	384 bits	384 bits	768 bits
Texture fill rate (texels/sec.)	43.2 GT/s	51.8 GT/s	42 GT/s	49.4 GT/s	77.7 GT/s
Performance (Gflops)	648	1063	1345	1581	2480

Processing capability of GPU vs. CPU



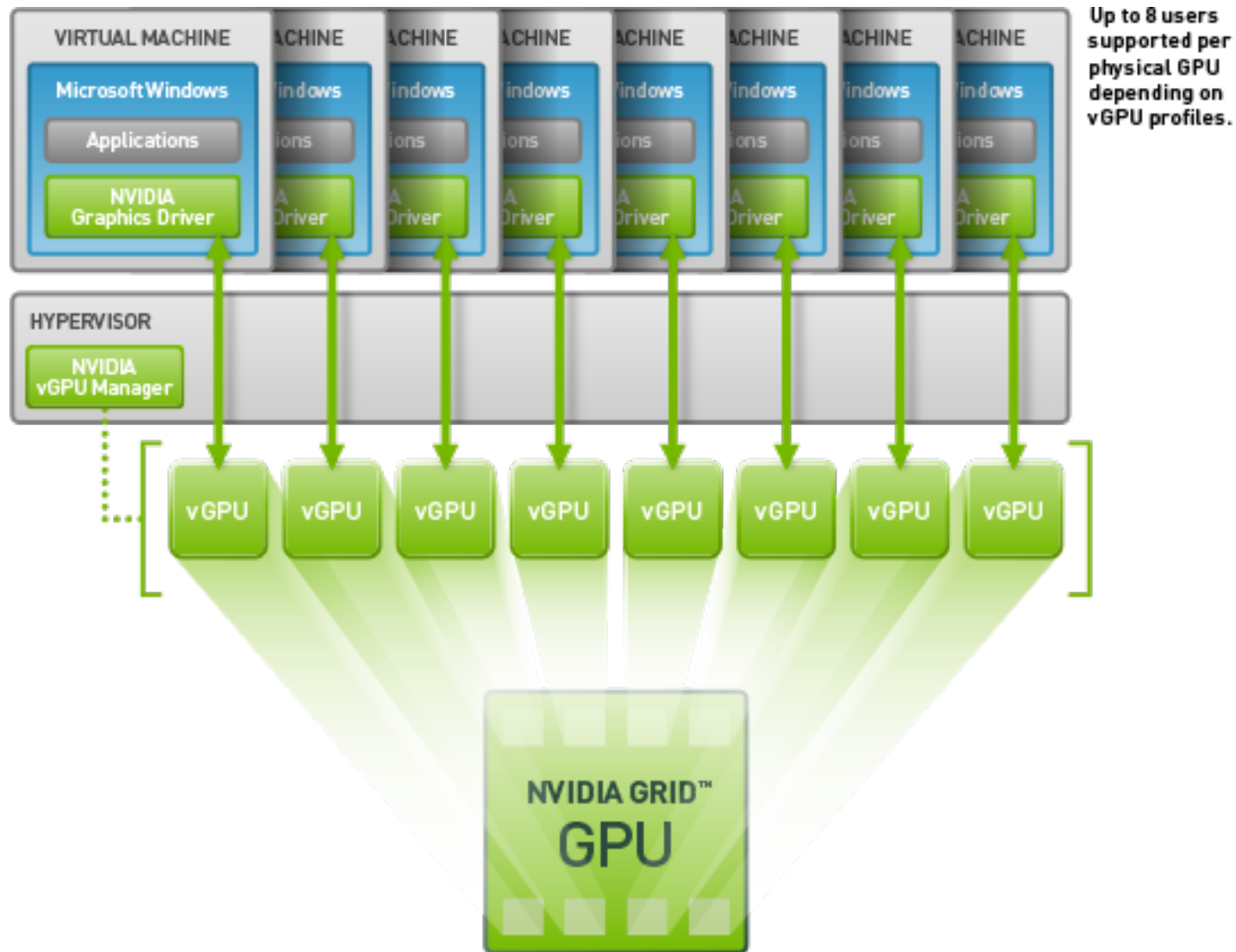
NVIDIA Grid Virtual GPU Technology

- ❑ Multiple users can share a single GPU, providing true PC performance and compatibility.
- ❑ Secure and graphics-intensive applications to designers and engineers wherever they are.
- ❑ Users access and use all their office productivity and design applications, virtually, from nearly any device—even on their own notebooks and portable devices—just as they would at their desks.
- ❑ For institutions in higher education, NVIDIA GRID vGPU enables scalable lab environments that deliver tools to students wherever they learn best.

Grid vGPU key features

	GRID K1	GRID K2
Number of GPUs	4 x KG107 GPUs	2 x GK104GPUs
Total NVIDIA CUDA cores	768 (192 cores/ GPU)	3072 (1536 cores/ GPU)
Total memory size	16 GB DDR3 (4 GB/ GPU)	8 GB GDDR5 (4 GB/ GPU)
Compatibiliy	DirectX 11, Shader 5.0, OpenGL 4.3	

Virtual GPU Technology

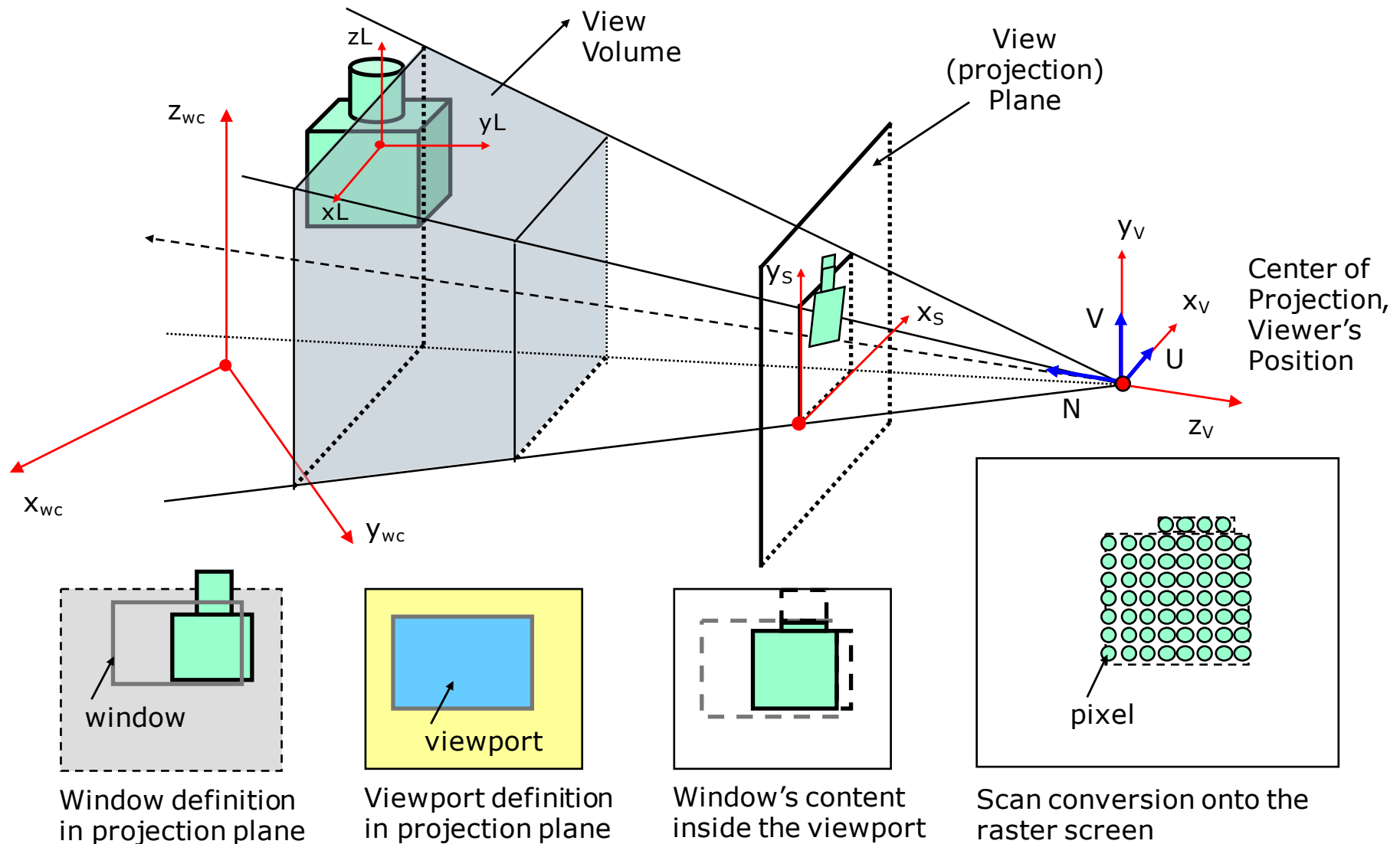


NVIDIA GeForce GTX 580

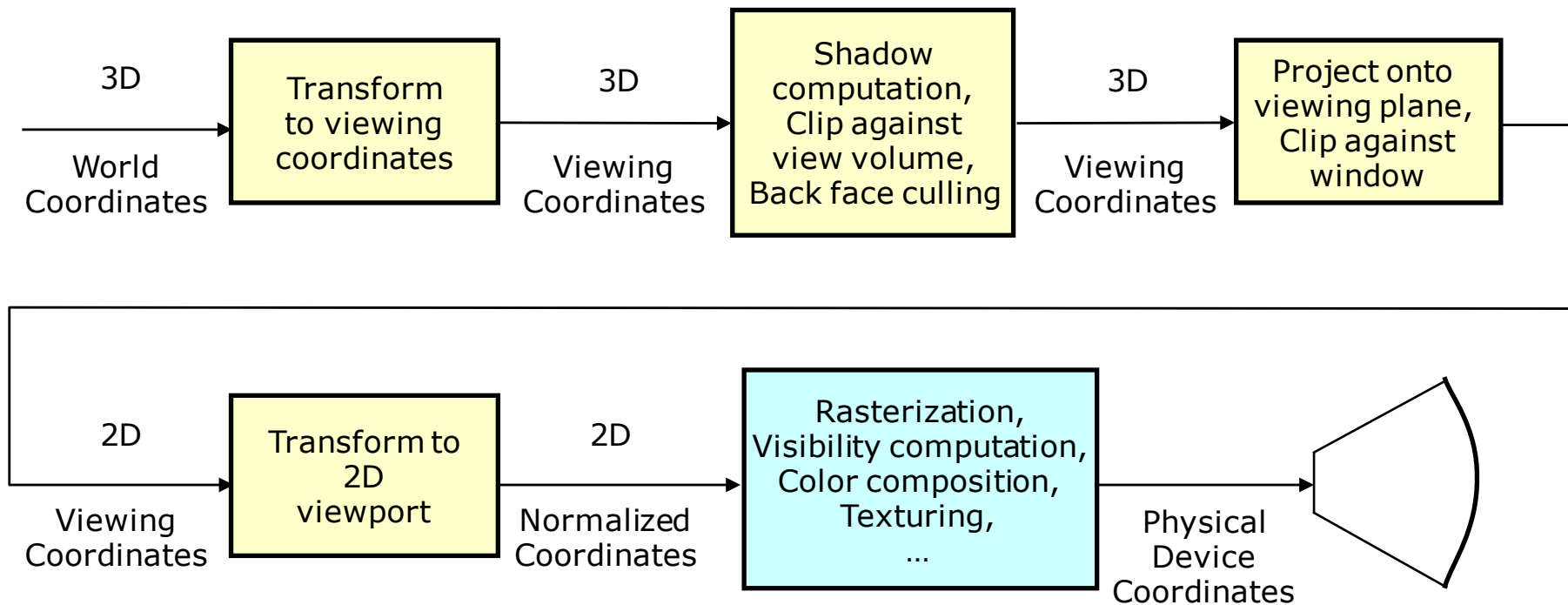


Picture taken from Wikipedia, the free encyclopedia.

Conceptual model of 3D viewing process



Viewing transformations pipeline



GPU pipeline

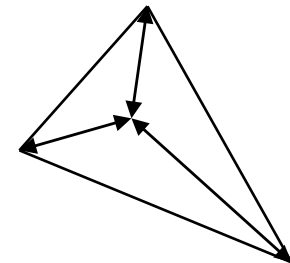
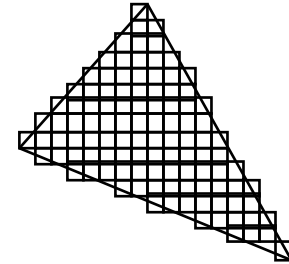
- ❑ GPU receives geometry information from the CPU as input and provides a picture as output
- ❑ Host interface:
 - Communication bridge between CPU and GPU.
 - Receives commands from the CPU and pulls geometry information from system memory.
 - Outputs a stream of vertices in object space with all their associated information (e.g. normals, texture coordinates, vertex color, ...)
- ❑ Vertex processing:
 - Receives vertices from the host interface in object space and outputs them in screen space.



GPU pipeline

□ Triangle setup:

- In this stage the triangles that are hidden or are outside the viewing frustum are removed
- Geometry information becomes raster information (screen space geometry is the input, pixels are the output)
- A fragment (virtual pixel) is generated if its center is inside the triangle
- Every fragment has its attributes computed to be the perspective correct interpolation of the three vertices that define the triangle (e.g. distance, z coordinate)



GPU pipeline

❑ Fragment processing:

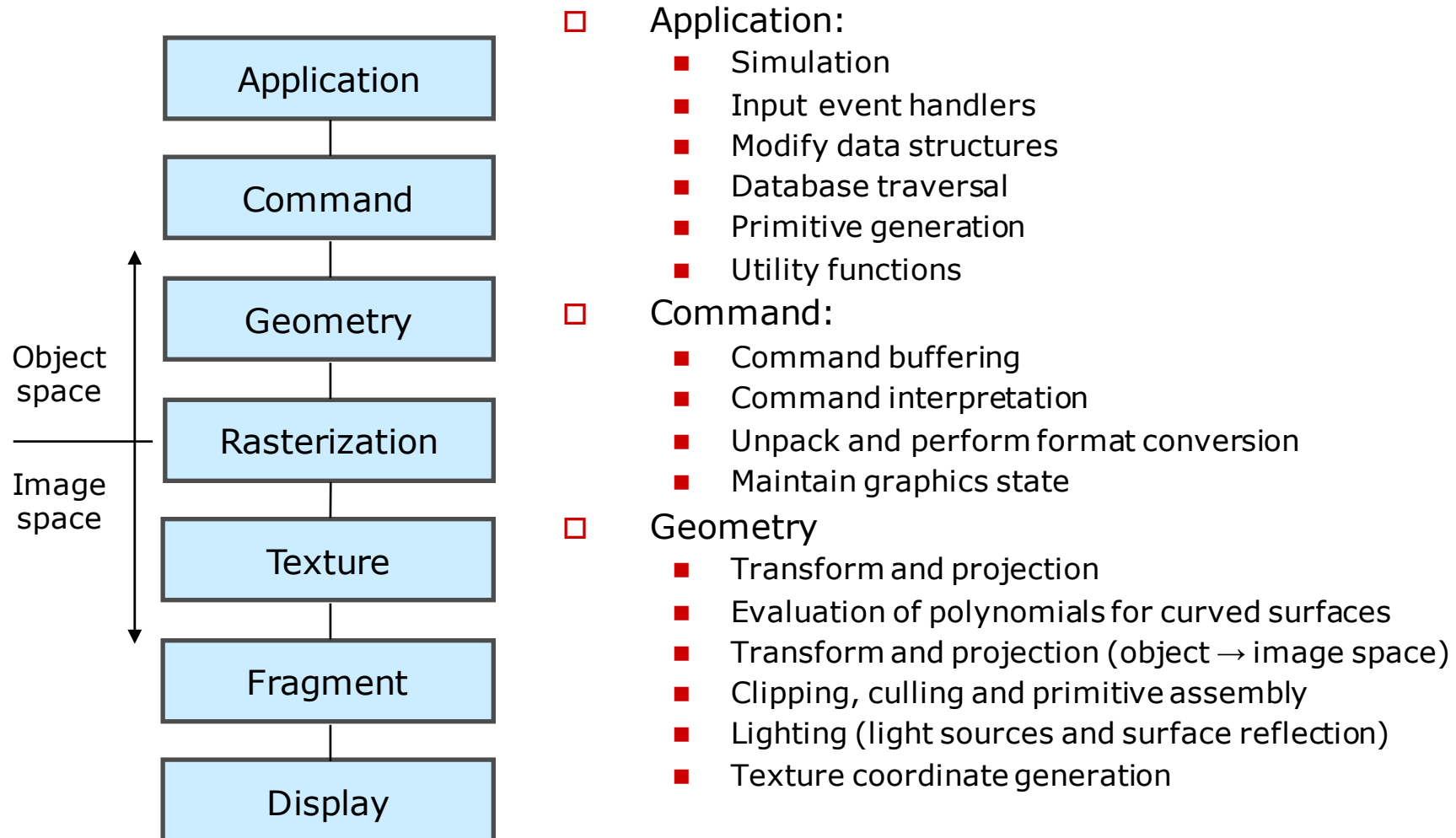
- Each fragment is a set of attributes (i.e. position, normal, texcoord, etc), which are processed to give the final color of the pixel
- Processing consists of texture mapping and mathematical operations
- !!! It is the bottleneck in graphics applications (e.g. games, animation, etc)

❑ Memory interface

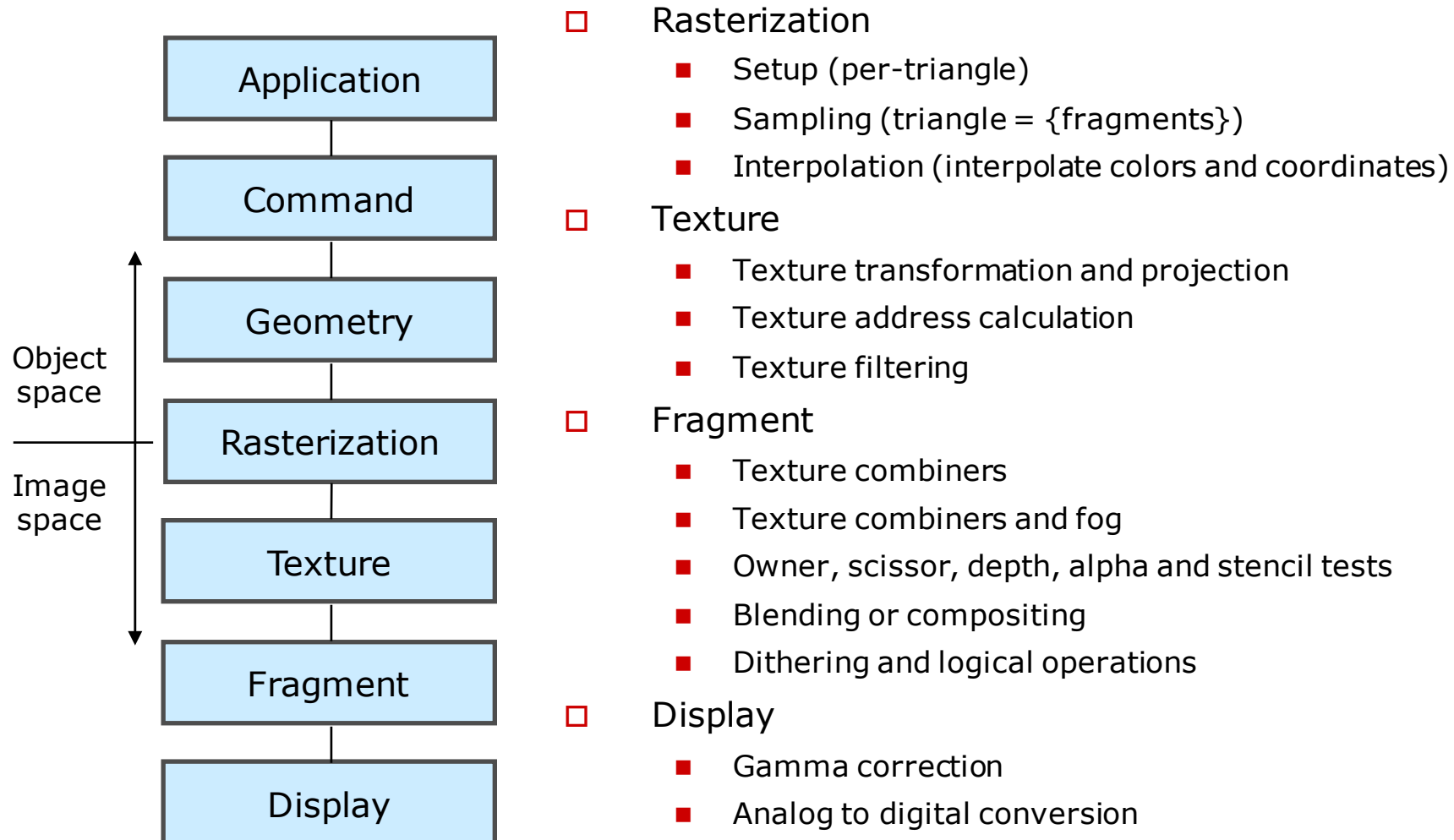
- Final fragment colors are written to the framebuffer
- A part of fragments are rejected by the zbuffer, stencil and alpha tests



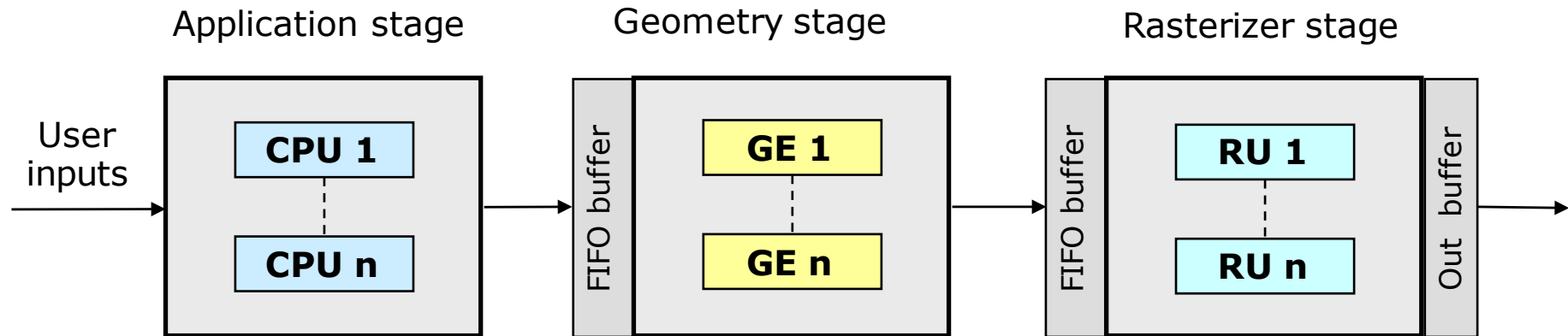
Canonical graphics pipeline



Canonical graphics pipeline



Graphics rendering pipeline



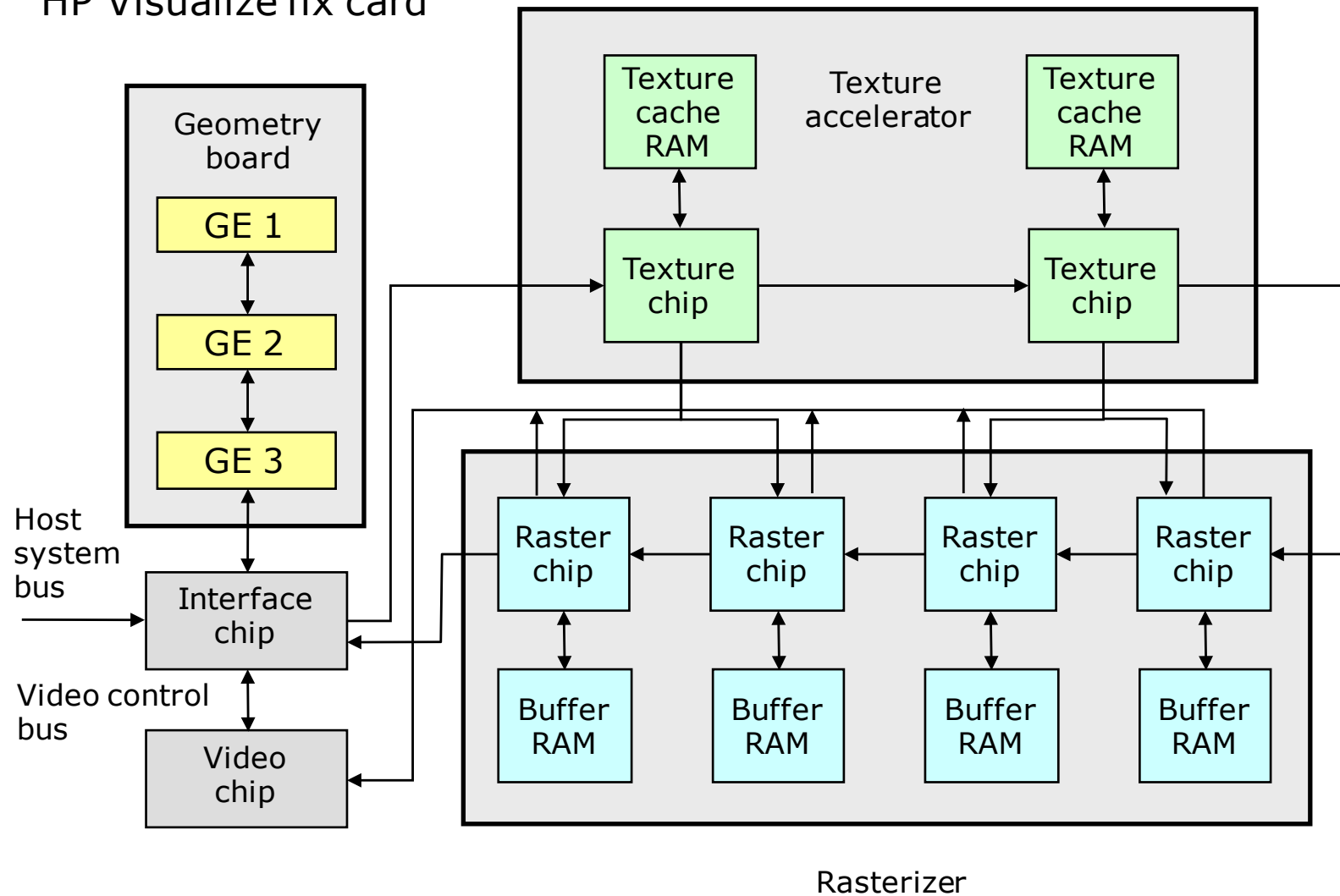
- | | | |
|--|--|--|
| <ul style="list-style-type: none">❑ Done in software.❑ Reads: world geometry state and user inputs.❑ May change the view to the simulation or change orientation of virtual objects. | <ul style="list-style-type: none">❑ Done in software or hardware.❑ Model transformations: translation, rotation, scaling;❑ Lighting computations.❑ Scene projection.❑ Slipping.❑ Mapping. | <ul style="list-style-type: none">❑ Done in hardware.❑ Converts vertex information (color and texture) into pixel information.❑ Performs antialiasing. |
|--|--|--|

Graphics rendering pipeline

- ❑ Parallel architectures for performance speed-up.
- ❑ Application stage – several CPUs.
- ❑ Geometric Stage – several GEs.
 - ❑ Each processing a portion of the scene geometry.
- ❑ Use FIFO buffers to store intermediate results.
- ❑ Each RU renders a patch of frame pixels.
- ❑ Frame buffer:
 - ❑ Pixel color info – color buffer.
 - ❑ Depth or z value – Z buffer.
- ❑ Modern architectures use double buffering:
 - ❑ Front buffer – stores the image to be displayed.
 - ❑ Back buffer – stores the new scene.
 - ❑ Then back buffer is displayed and the new image is rendered in the front buffer – flicker is reduced.

Rendering pipeline example

HP Visualize fix card



Graphics pipeline bottlenecks

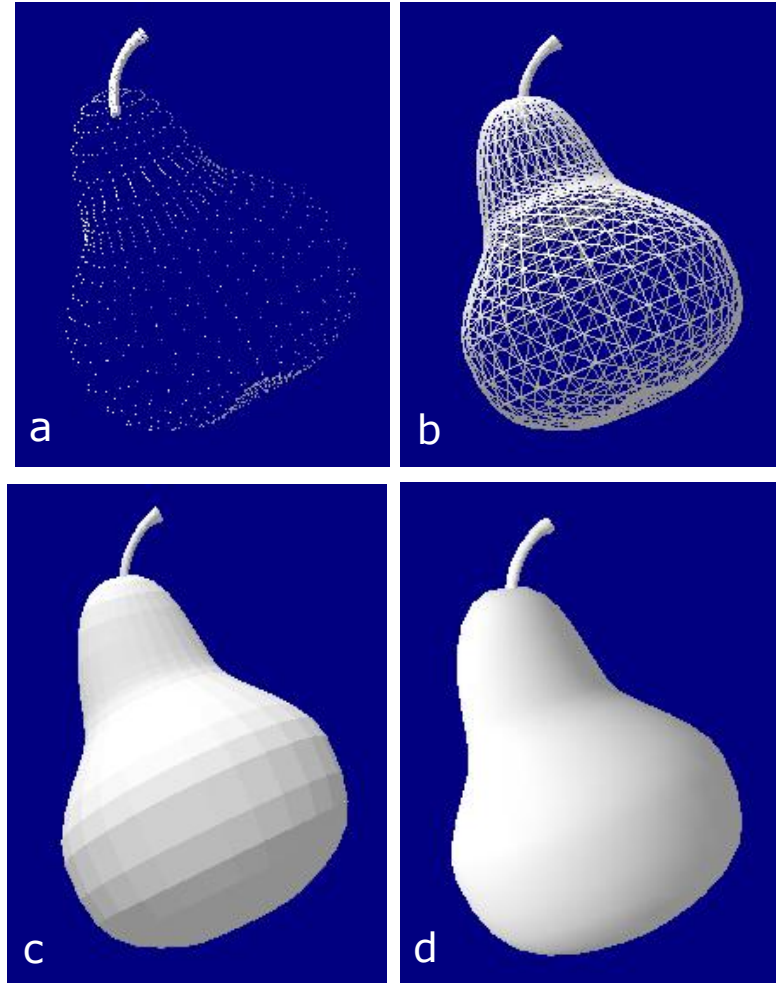
- ❑ One of the 3 stages will be the slowest
 - ❑ Operates at 100%
 - ❑ Set the overall pipeline processing time
- ❑ Bottleneck appears in the
 - ❑ Application Stage (most cases)
 - ❑ Geometry stage
 - Ex. if the frame refresh rate increases when the number of light sources is reduced. Transform-limited pipeline
 - ❑ Rasterizing stage
 - Ex. if pipeline output increases when the size or the resolution of the display window is reduced. Fill-limited pipeline

Optimizations in Application stage

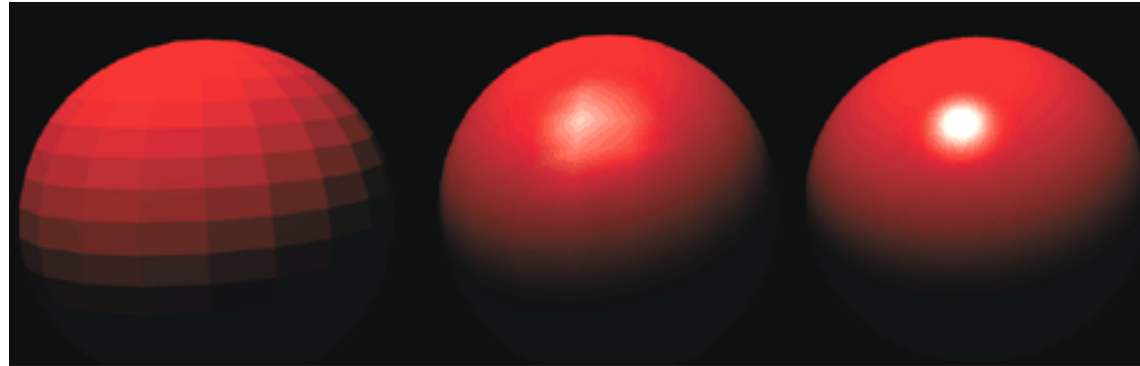
- ❑ Replace CPU with a faster one.
- ❑ Add another CPU.
- ❑ Reduce the load of the CPU
 - ❑ reduce scene complexity
 - ❑ 3D models with lower polygonal number
- ❑ Optimize the simulation software.
- ❑ Use lower precision arithmetic instead of double precision.
- ❑ Minimize the number of divisions in the code.

Optimizations in Geometry stage

- ❑ Polygon shading modes
 - (a) dot modeling
 - (b) wireframe.
 - (c) faceted or flat shading.
Color in polygon is constant.
 - (d) Gouraud shading.
Intensity interpolation.
- ❑ Phong shading
- ❑ Texturing
- ❑ Computational load increases.
Frame refresh rate decreases.
- ❑ Optimize the pipeline by using the type of polygon for which its rendering hardware is optimized.



Optimizations in Geometry stage



Flat

Gouraud

Phong

Wireframe

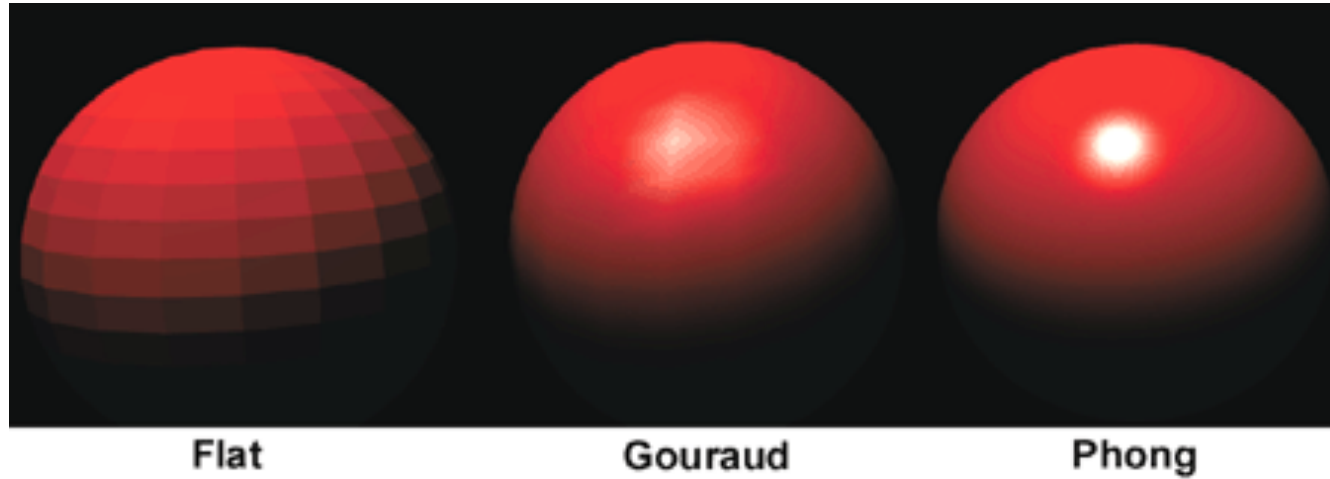
Flat



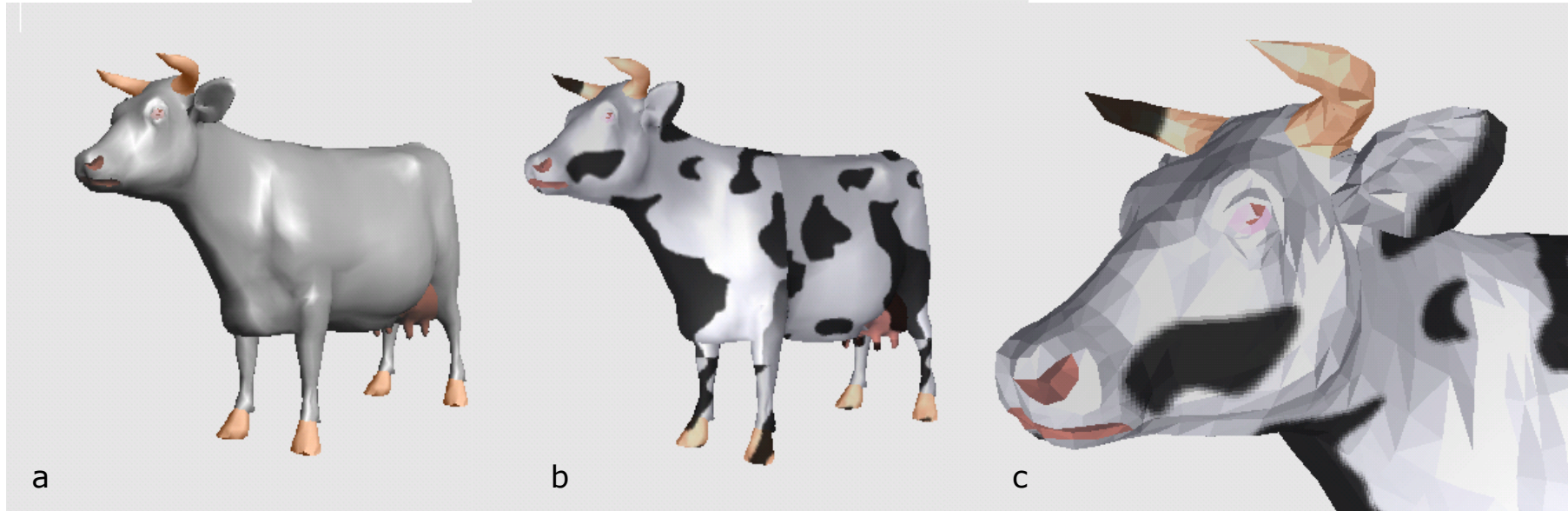
Gouraud

Phong

Optimizations in Geometry stage



Optimizations in Geometry stage

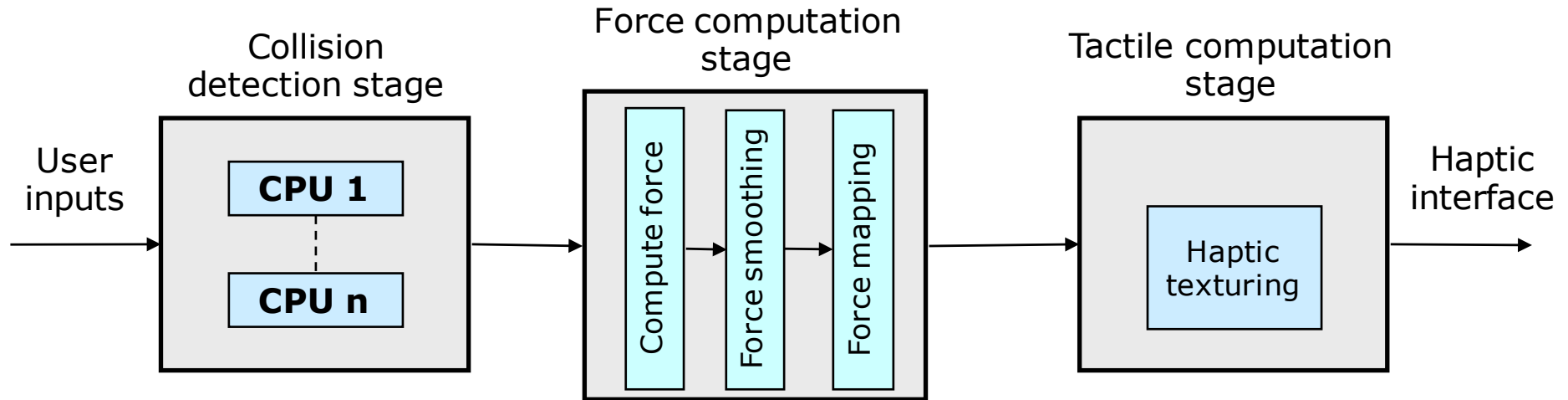


(a) Phong shading. (b) Phong shading and texture. (c) Flat shading and texture.

Optimizations in Rasterizing stage

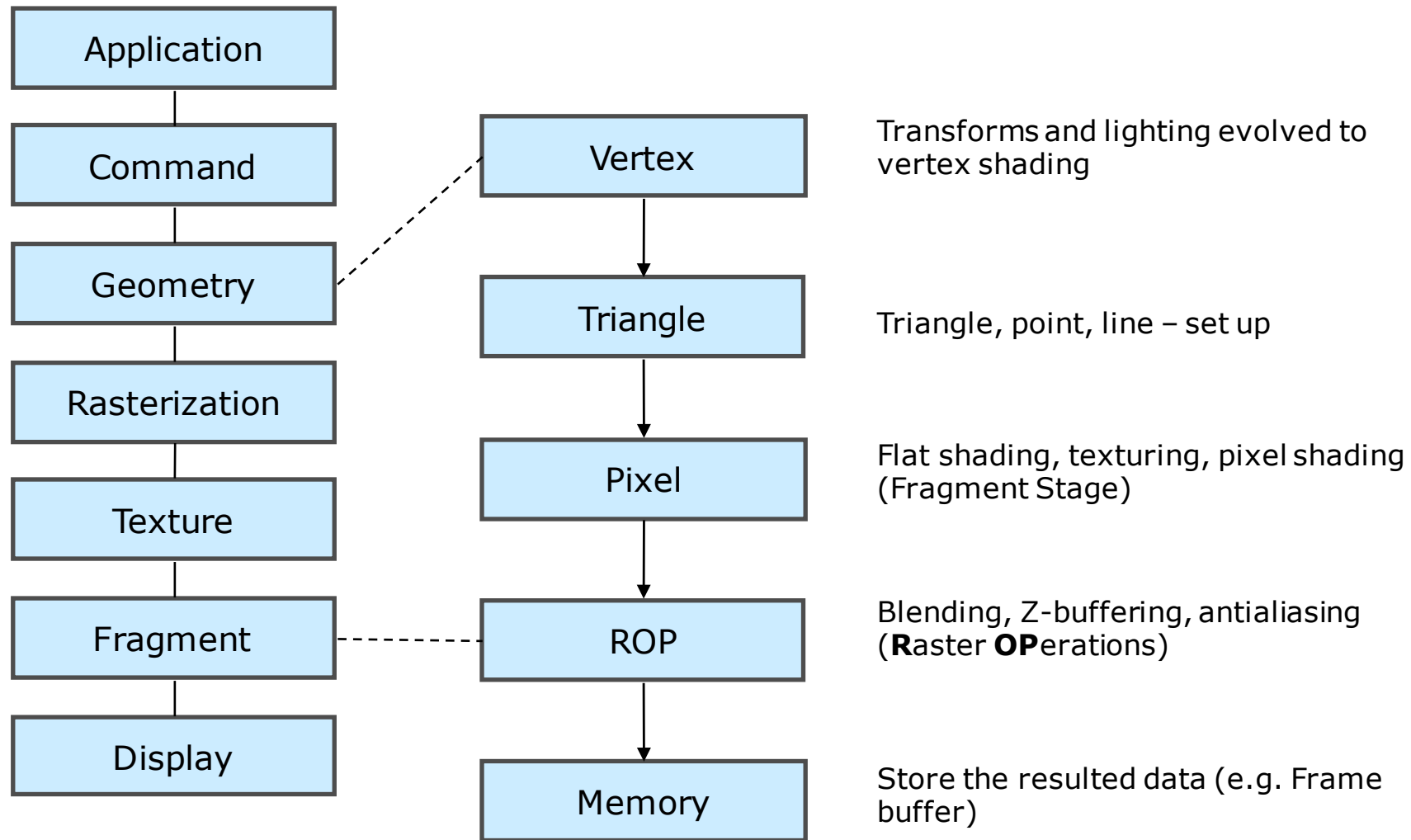
- ❑ Reduce the size of the display window.
- ❑ Reduce the resolution
 - ❑ Less work for the RUs.
- ❑ Assume that RU works at 100% and the other stages work at 80%
 - ❑ Increase the model complexity - in order to balance the load.
- ❑ If GEs operate at 100%
 - ❑ Increase display resolution or size to balance load.

Haptics rendering pipeline

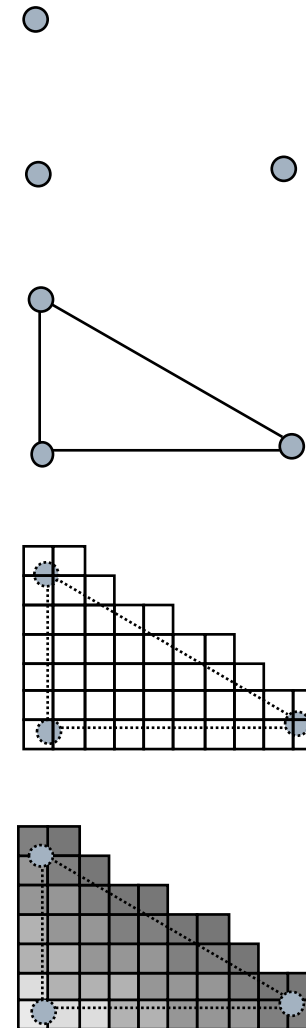
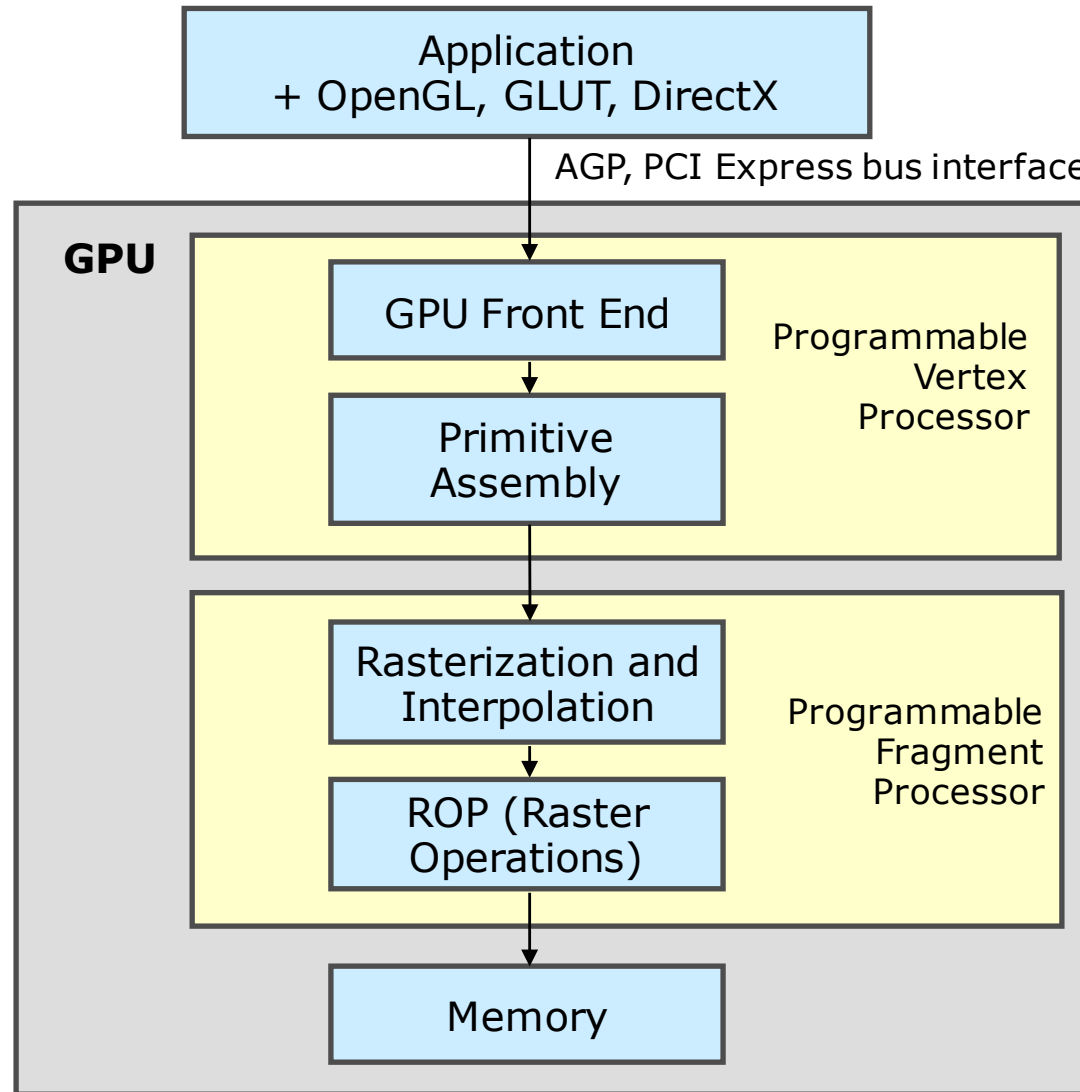


- ❑ Collision detection to determine which virtual objects collide.
- ❑ Only the colliding objects in the scene are passed to next pipeline stages.
- ❑ The haptics rendering pipeline has a less standardized architecture, i.e. it reflects the evolving state of the haptic technology.
- ❑ Compute friction forces (Hooke's law), the contact force has a spring like dependence on the degree of surface deformation. Other models – damping and friction forces.
- ❑ Simpler force models optimize the pipeline.
- ❑ Force smoothing – adjust the direction of the force vector, in order to avoid sharp transitions between polygonal surfaces. Force mapping projects the force to the characteristic of the haptic display system.
- ❑ Renders the touch feedback component of the simulation.
- ❑ Computed effects: surface temperature, vibrations – added to the force vector and sent to the haptic output display.

Classic GPU pipeline (Processor/function)



GPU pipeline (Function blocks)



GPU graphics pipeline

- GPU has evolved from *hardwired* implementation of the graphics pipeline to a *programmable* computational structure.

- Fixed-function units for transforming vertices and texturing pixels have been subsumed by a *unified grid of processors*, or *shaders*.

Ex. NVIDIA GeForce 3, launched in February 2001, introduced *programmable vertex shaders*. GeForce 3 introduced the reconfigurability in pixel processing, exposing the texturing hardware's functionality as a set of *register combiners* that could achieve novel visual effects.

- The ATI Radeon 9700 (July 2002) and NVIDIA GeForce FX (January 2003) replaced the register combiners with fully *programmable pixel shaders*. NVIDIA GeForce 8800 (November 2006), adds programmability to the primitive assembly stage, allowing developers to control how they construct triangles from transformed vertices.
- *Color*. The traditional graphics pipeline provided only 8-bit integers per color channel (range 0 to 255). The ATI Radeon 9700 increased the representable range of color to 24-bit floating point, and NVIDIA GeForce FX followed with both 16-bit and 32-bit floating point.

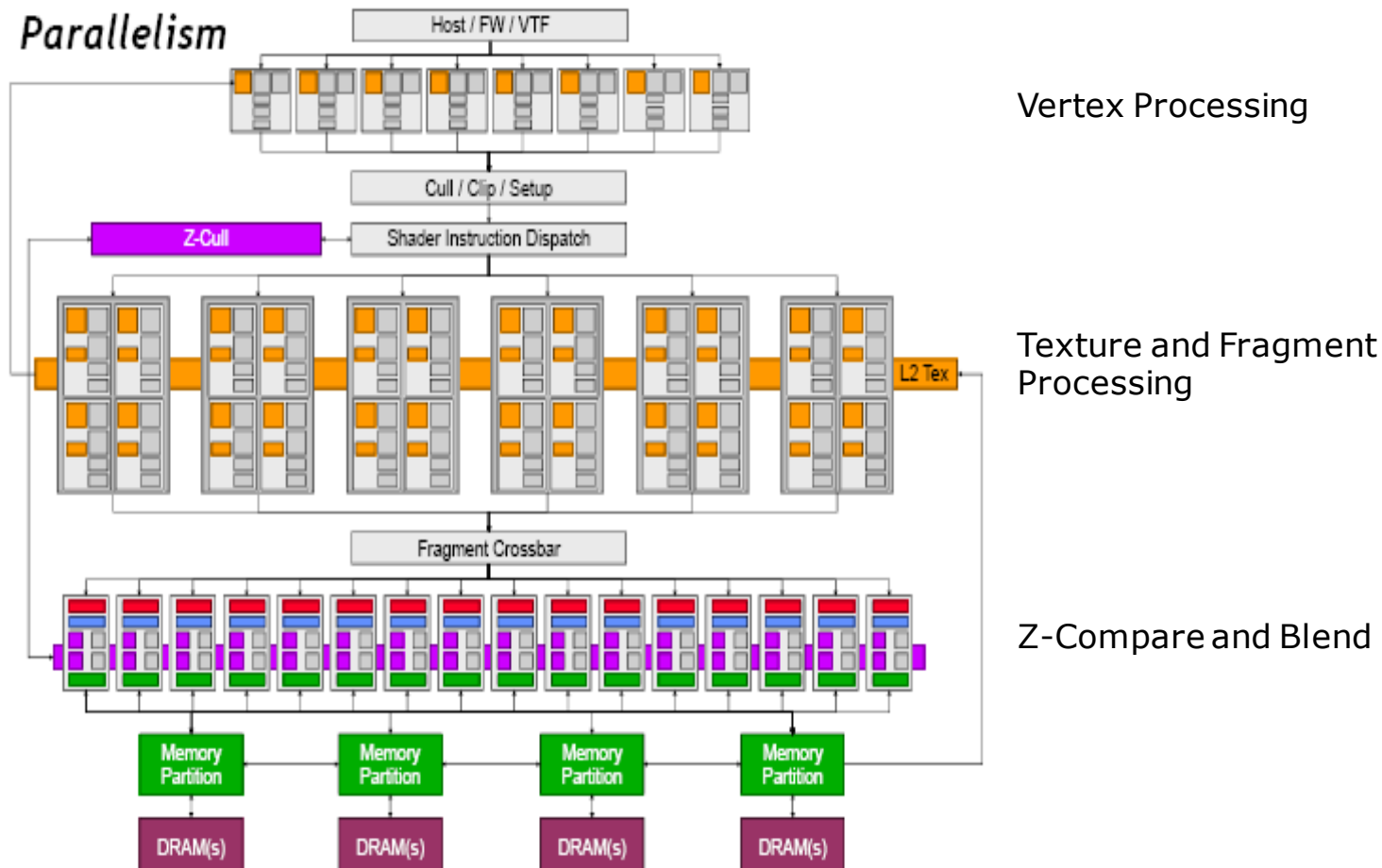
GPU graphics pipeline

- The number of the four-wide vector processors executing in parallel has increased from only four on GeForce FX to 16 on GeForce 6800 (April 2004) to 24 on GeForce 7800 (May 2005). The GeForce 8800 includes 128 scalar *shader processors (Streaming Processors)*.
- The NVIDIA GeForce 8800 GPU replaces the traditional graphics pipeline with a *unified shader architecture* in which vertices, triangles, and pixels recirculate through a set of programmable processors. The flexibility and computational power of these processors support the general-purpose computing tasks. A single GeForce 8800 chip achieves a sustained 330 billion floating-point operations per second (Gflops) on simple benchmarks

NVIDIA GeForce 7800

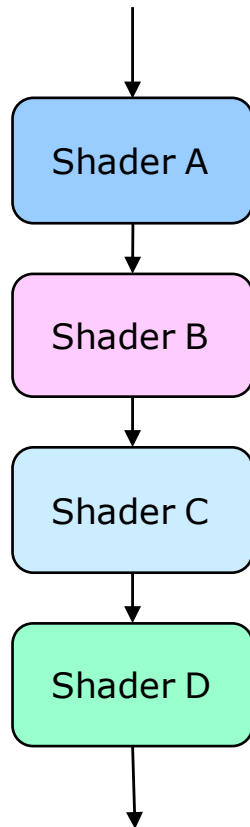
GeForce 7800 Parallelism

Figure courtesy of NVIDIA

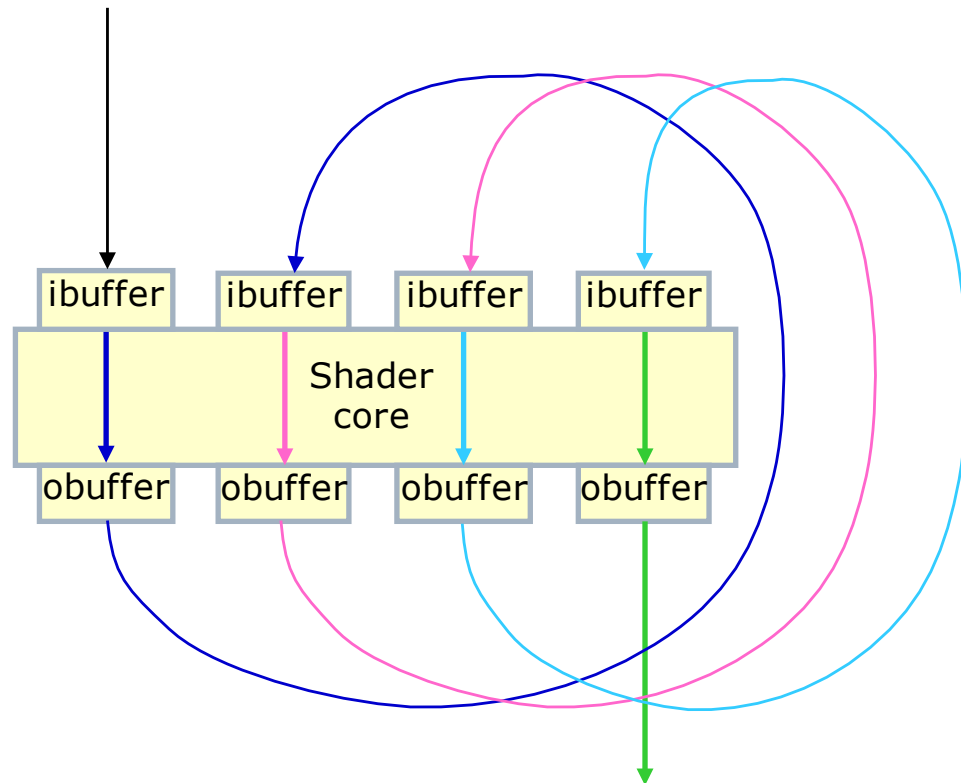


Classic vs. unified shader architecture

Discrete Design

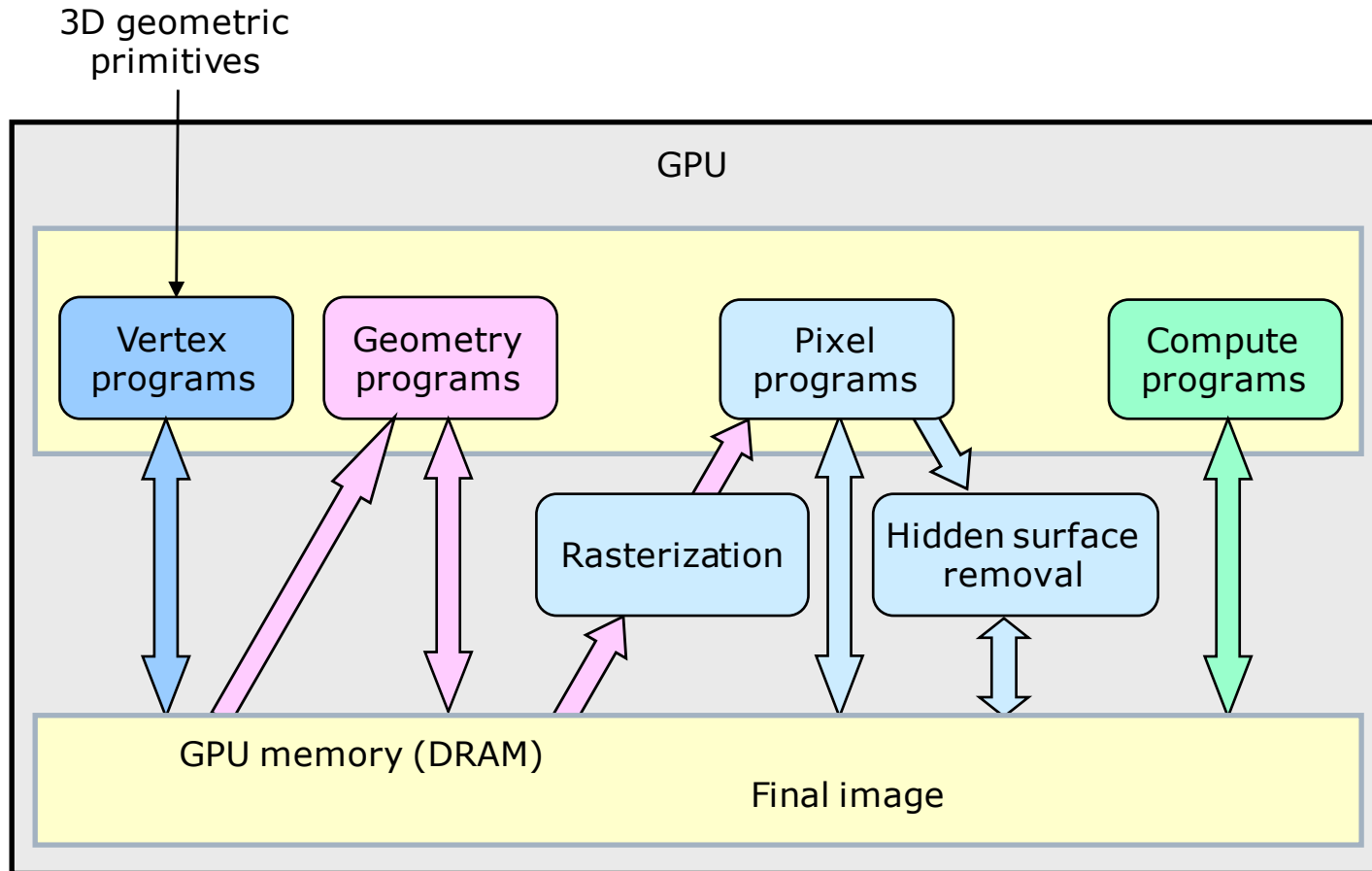


Unified Design



NVIDIA GeForce 8800 GPU Architecture Overview, November 2006 TB-02787-001_v01, <http://www.nvidia.com>

Unified shader architecture



Ref: David Luebke, NVIDIA Research, Greg Humphreys, University of Virginia, "How GPUs Work"

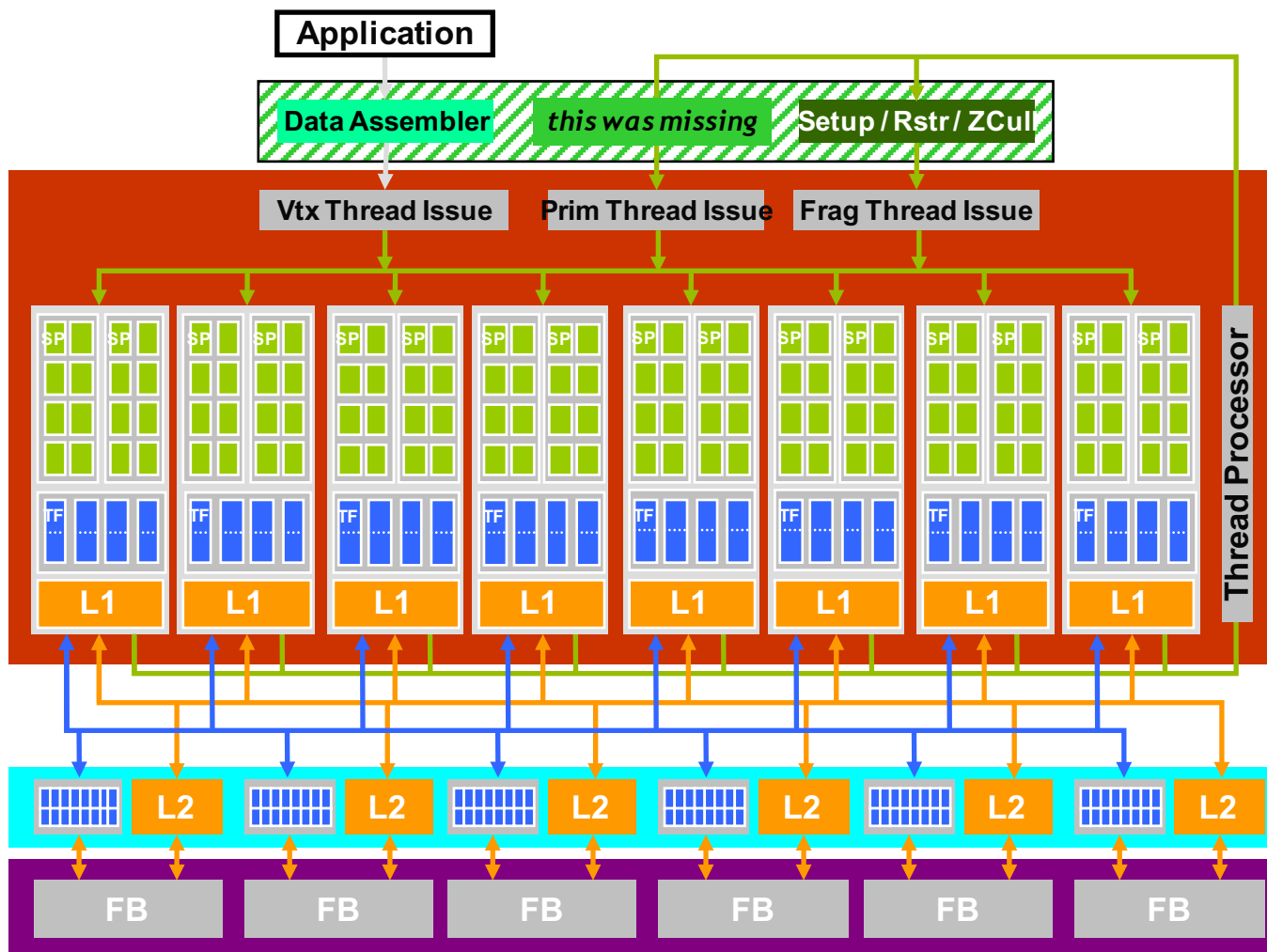
NVIDIA GeForce 8800

GeForce 8800 block diagram (NVIDIA Corporation 2006)

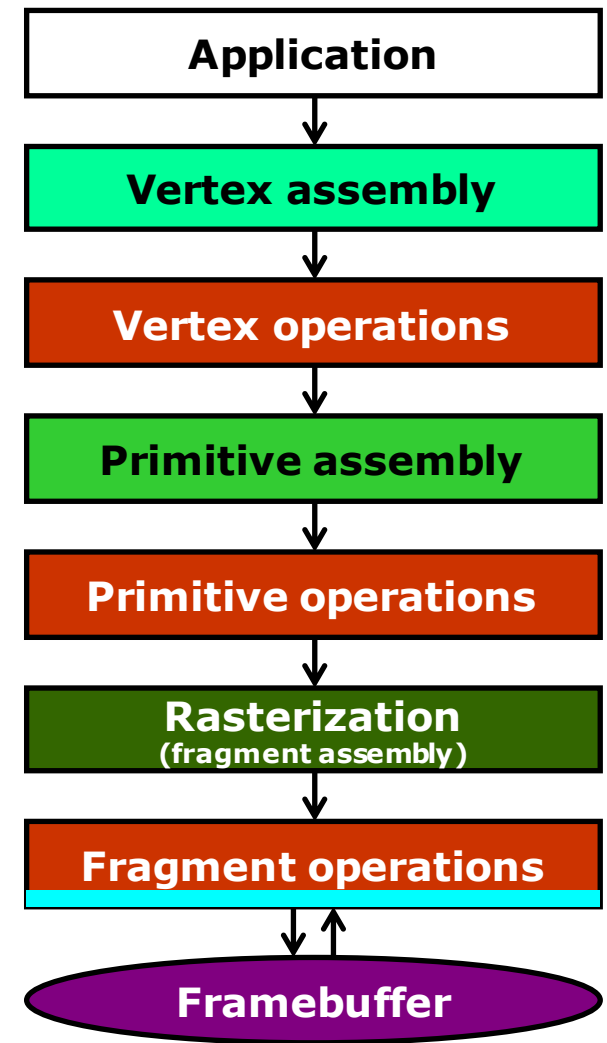


GeForce 8800	Core Clock (MHz)	Shader Clock (MHz)	Memory Clock (MHz)	Memory Amount	Memory Interface	Memory Bandwidth (GB/sec)	Texture Fill Rate (billion/sec)
Ultra	612	1500	1080	768 MB	384 bit	103.7	39.2
GTX	575	1350	900	768 MB	384 bit	86.4	36.8
GTS	500	1200	800	640 MB	320 bit	64	24

NVIDIA GeForce 8800

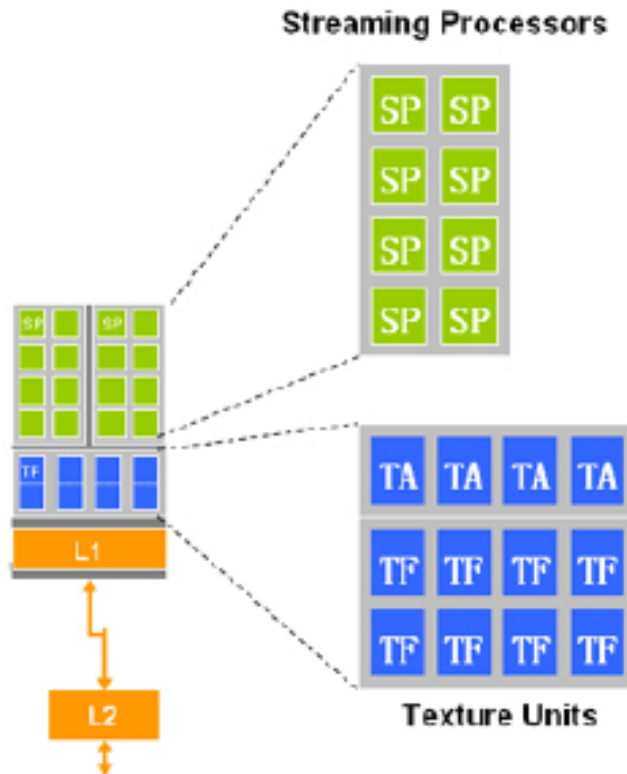


NVIDIA GeForce 8800



OpenGL Pipeline

Streaming processors, texture units, and caches



SP – Stream Processors

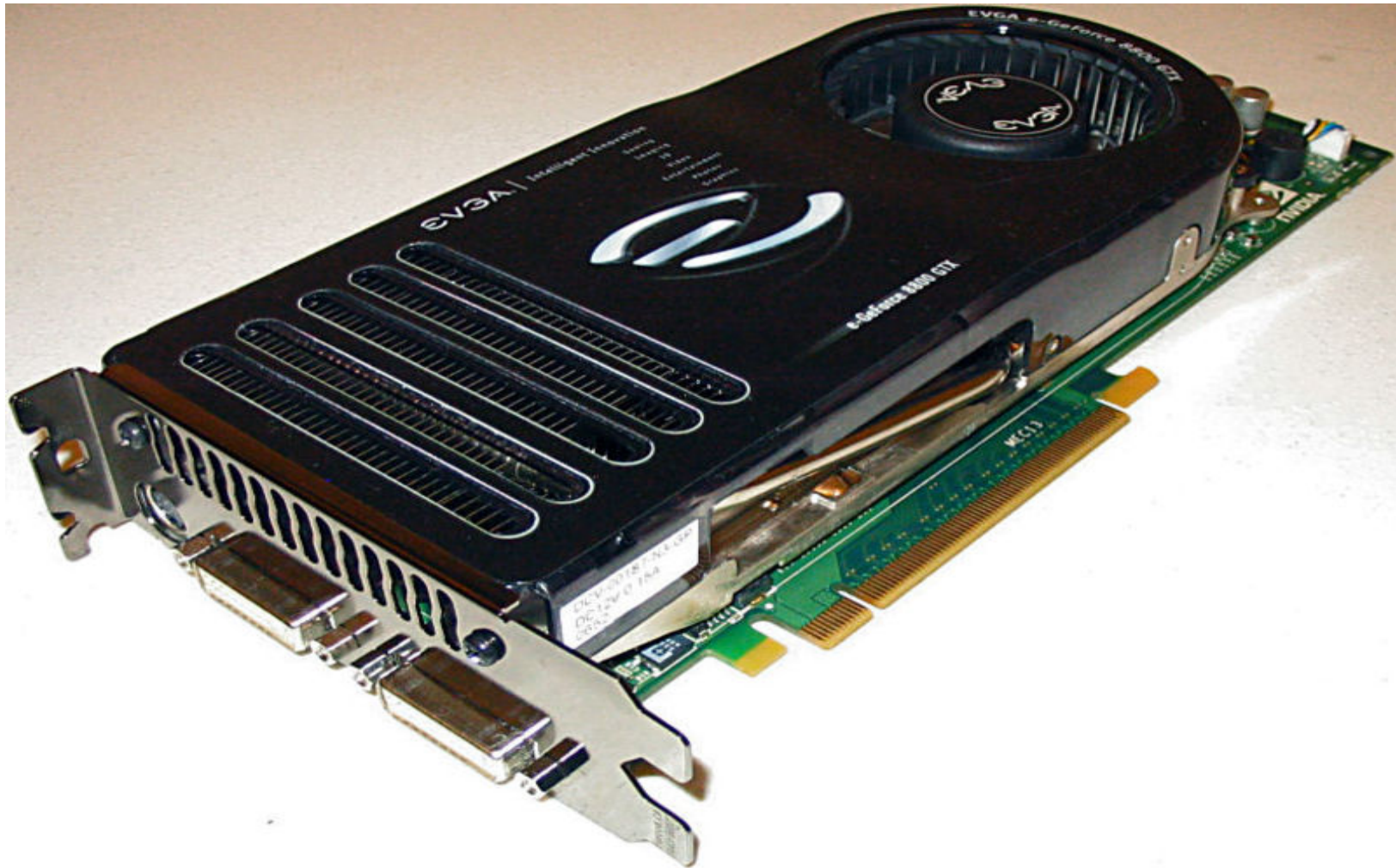
TF – Texture Filtering Unit

TA – Texture Address Unit

L1/L2 – Caches

- Stream processors perform calculations on an input stream and produce an output stream that can be used by other stream processors. SPs can be grouped in close proximity, and in large numbers, to provide immense parallel processing power.
- SPs are a critical component of NVIDIA GigaThread technology, where thousands of threads can be in flight within a GeForce 8800 GPU at any given instant. SPs schedule and dispatch various types of threads for execution (e.g. pixel, vertex, geometry, and physics).

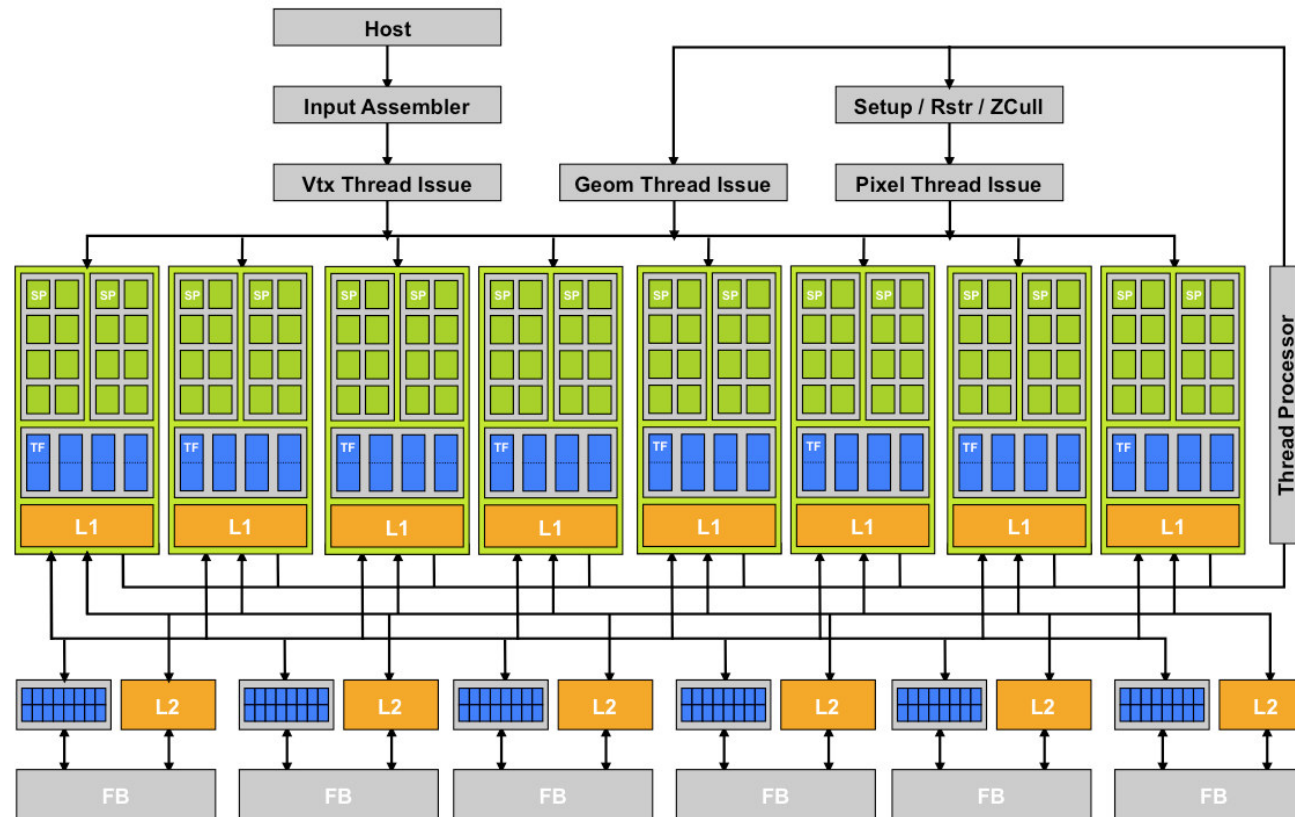
NVIDIA GeForce 8800 GTX



Picture taken from Wikipedia, the free encyclopedia.

GPGPU – General Purpose GPU

- Modern architecture – NVIDIA G80 – Unified Shader



http://www.nvidia.com/content/PDF/fermi_white_papers/P.Glaskowsky_NVIDIAFermi-TheFirstCompleteGPUComputingArchitecture.pdf

- GPGPU – Specialized languages (CUDA, OpenCL)

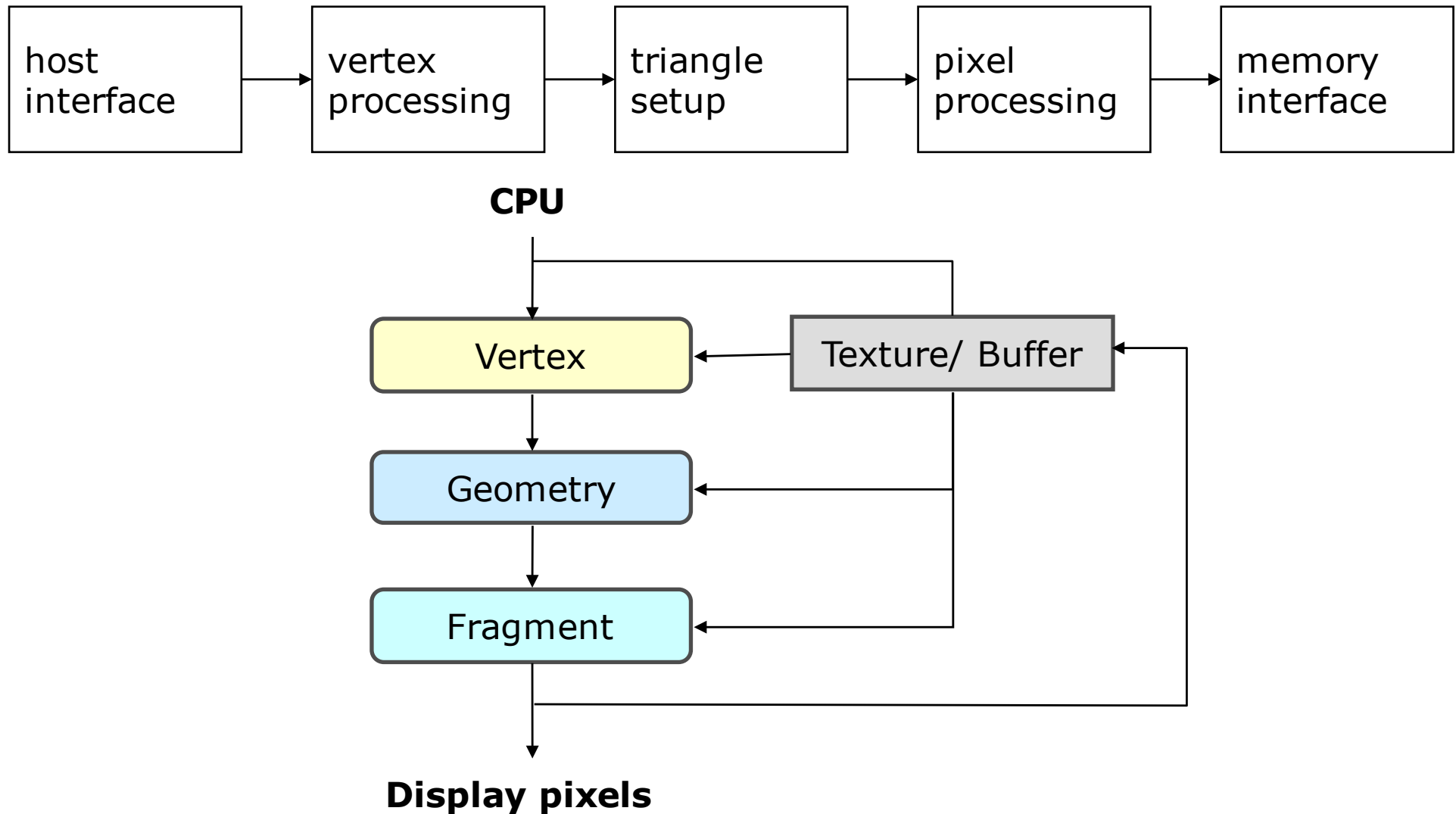
GPGPU

- GPGPU - General Purpose Graphics Processing Unit
- Application domains:
 - General computation
 - Linear algebra
 - Differential equations
 - Image processing and Computer vision
 - Pattern recognition
 - Classification

Shader languages

- ❑ Assembler
- ❑ Cg (NVIDIA)
C for Graphics
- ❑ GLSL (OpenGL)
OpenGL Shading Language
- ❑ HLSL (Microsoft/Direct3D)
High Level Shading Language

GPU computation pipeline (programmer's view)



Cg Language

- ❑ C for Graphics (Cg) is a high-level shading language developed by NVIDIA's open-source high-level shading language.
- ❑ Developed in collaboration with Microsoft for *programming vertex and pixel shaders*.
- ❑ Cg is based on the C programming language. New data types were added to make Cg more suitable for programming graphics processing units.
- ❑ Cg code is portable to a wide range of hardware and platforms
- ❑ Cg can be used with two APIs: OpenGL or DirectX. Each has its own set of Cg functions to communicate with the Cg program
- ❑ Cg compiler outputs DirectX or OpenGL shader programs.

Ref: Cg Homepage, http://developer.nvidia.com/page/cg_main.html

GLSL – OpenGL Shading Language

- ❑ GLSL (OpenGL Shading Language), also known as GLSLang, is a high level shading language based on the C programming language. It was created by the OpenGL ARB to give developers more direct control of the graphics pipeline without having to use assembly language or hardware-specific languages.
- ❑ The OpenGL Shading Language is actually two closely related languages. These languages are used to create shaders for the programmable processors contained in the OpenGL processing pipeline.
- ❑ The specific languages are referred by the name of the processor they target: vertex or fragment.
- ❑ The *vertex processor* is a programmable unit that operates on incoming vertices and their associated data. Compilation units written in the OpenGL Shading Language to run on this processor are called *vertex shaders*.
- ❑ The *fragment processor* is a programmable unit that operates on fragment values and their associated data. Compilation units written in the OpenGL Shading Language to run on this processor are called *fragment shaders*.

Ref: GLSL specification, <http://www.opengl.org/documentation/glsl/>

HLSL - High Level Shading Language

- High Level Shading Language (HLSL) is a proprietary shading language developed by Microsoft for use with the Microsoft Direct3D API.
 - It is analogous to the GLSL shading language used with the OpenGL standard. It is very similar to the NVIDIA Cg shading language, as it was developed alongside it.
- There are three HLSL program forms: *vertex shaders*, *geometry shaders*, and *pixel (or fragment) shaders*.
- A *vertex shader* can be used for:
 - Transform vertices to clip-space
 - Pass normal, texture coordinates or other vertex data to Pixel Shader
 - Transforming vectors to other spaces (e.g. texture space)
 - Calculate vertex lighting
 - Distort geometry (e.g. waves, fish-eye camera)
- The Pixel Shader can be used for:
 - Texturing objects
 - Pixel lighting
 - Pixel shadow
 - Environment mapping
 - Normal mapping (every pixel has its own normal)

Ref: High Level Shading Language for DirectX, [http://msdn2.microsoft.com/en-us/library/bb509561\(VS.85\).aspx](http://msdn2.microsoft.com/en-us/library/bb509561(VS.85).aspx)

Software technologies

- ❑ OS: Windows, Linux, Mac
- ❑ API: OpenGL, DirectX, CUDA C/C++, OpenCL, DirectCompute, CUDA Fortran, and others
- ❑ Language: Cg, GLSL, HLSL
- ❑ Compiler: Cg, OpenGL, DirectX, ASHLI
- ❑ Runtime: CgFX, OSG, ASHLI, other codes

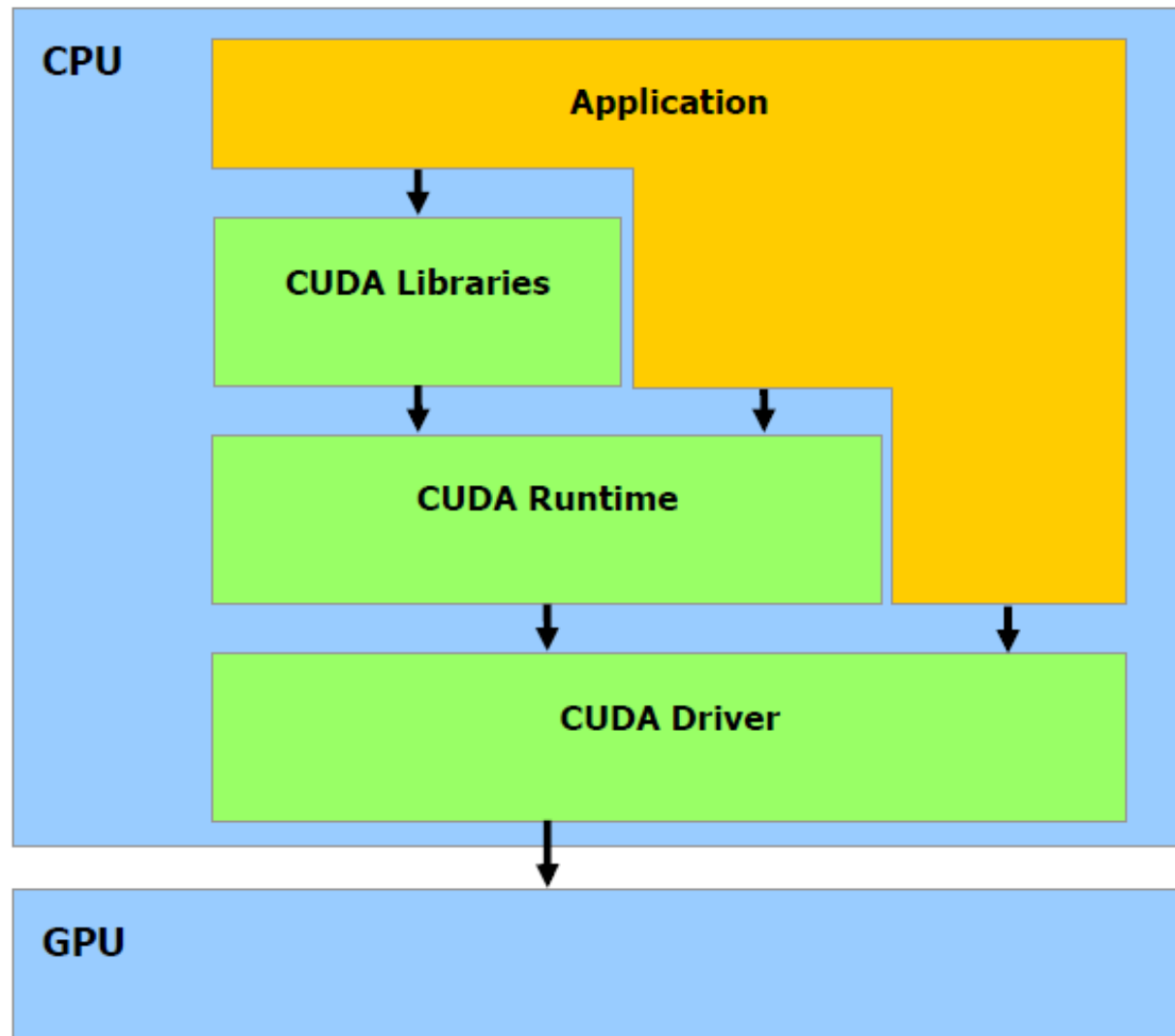
CUDA

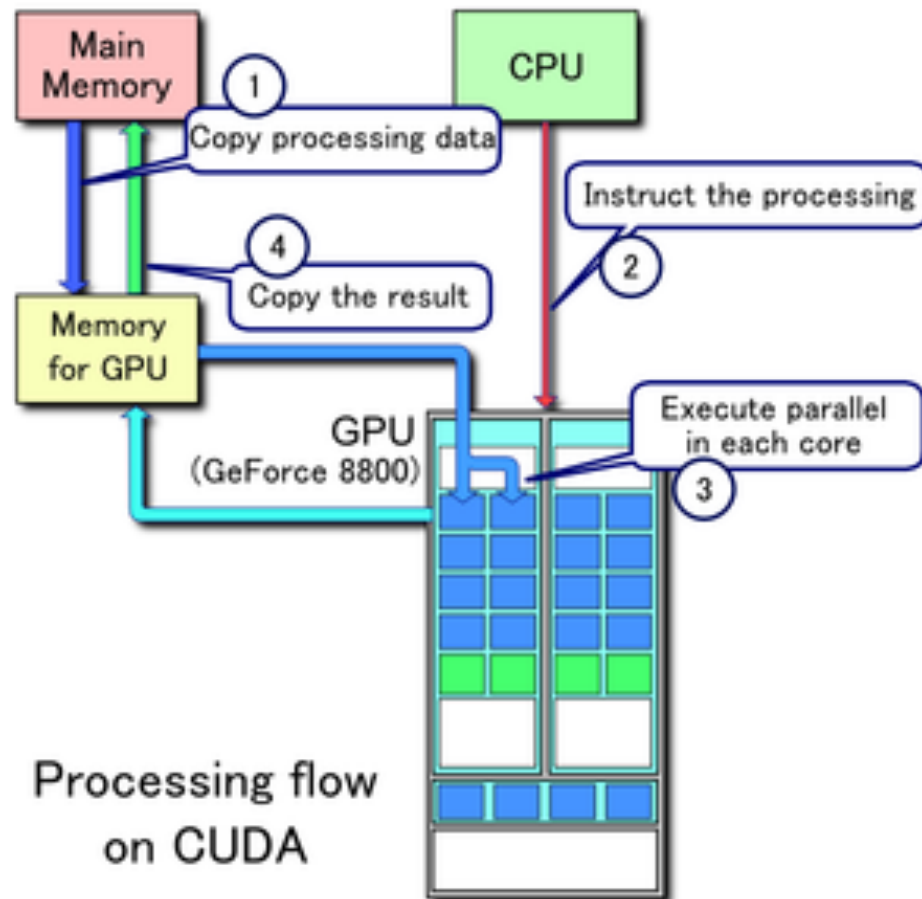
- ❑ CUDA - Compute Unified Device Architecture
- ❑ NVidia GPU cards support an API extension to the C programming language CUDA
- ❑ CUDA allows specified functions from a normal C program to run on the GPU's stream processors
- ❑ C programs become capable of taking advantage of the GPU's ability to operate on large matrices in parallel, while still making use of the CPU when appropriate
- ❑ CUDA is the first API to allow CPU-based applications to access directly the resources of a GPU for more general purpose computing without the limitations of using a graphics API

Reference:

1. CUDA site, http://www.nvidia.com/object/cuda_home.html
2. CUDA on Wikipedia, <http://en.wikipedia.org/wiki/CUDA>

CUDA Software Stack





Example of CUDA processing flow [Ref 2]:

1. Copy data from the main to the GPU memory
2. CPU instructs the process to GPU
3. GPU execute parallel in each core
4. Copy the result from GPU to the main memory

OpenCL

- ❑ OpenC - Open Computing Language
- ❑ Cross platform environment for general purpose computing on GPU
- ❑ Low-level API for heterogeneous computing that runs on CUDA architecture GPUs
- ❑ C-like programming language to work with massive parallel computing power of NVIDIA GPU's to create compelling computing applications
- ❑ OpenCL supports processors from other vendors
- ❑ First conformant GPU implementation for both Windows and Linux in 2009

Reference:

- <http://developer.nvidia.com/ocl>

OpenCL and CUDA

- ❑ OpenCL is portable between different graphics hardware
- ❑ CUDA is NVIDIA only
- ❑ CUDA is an independent language and is much more powerful and has a set of good tools. CUDA coding environment is almost like coding in C
- ❑ OpenCL is easier to use out of the box
- ❑ Both have extensive documentation and examples
- ❑ CUDA allows for greater control, hence can be better fine-tuned for higher performance

Reference:

- http://www.cse.scitech.ac.uk/disco/mew20/presentations/GPU_MattHarvey.pdf

Distributed visualization

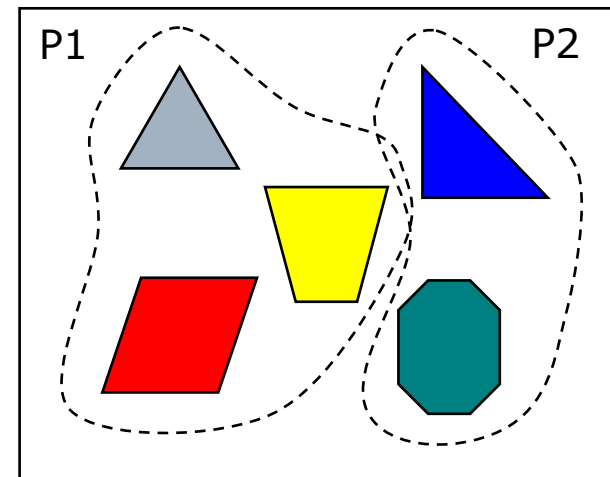
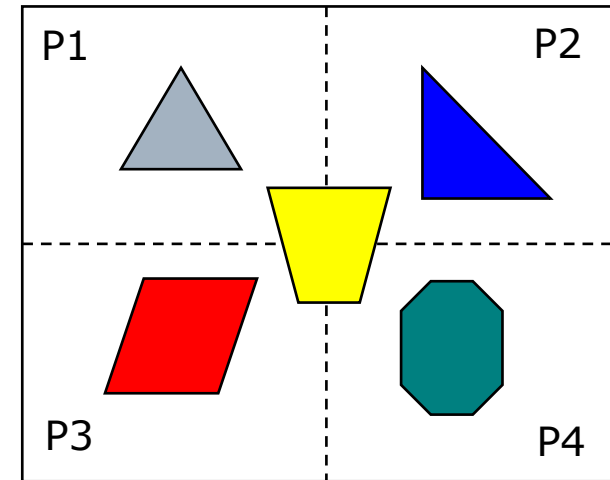
- Parallel and distributed computation in visualization works in:

1. Image space

- decomposition of a visualization task into a collection of sub-tasks, each responsible for a small portion of pixels in the visualization image to be synthesized.
- *sort-first* algorithms

2. Object space

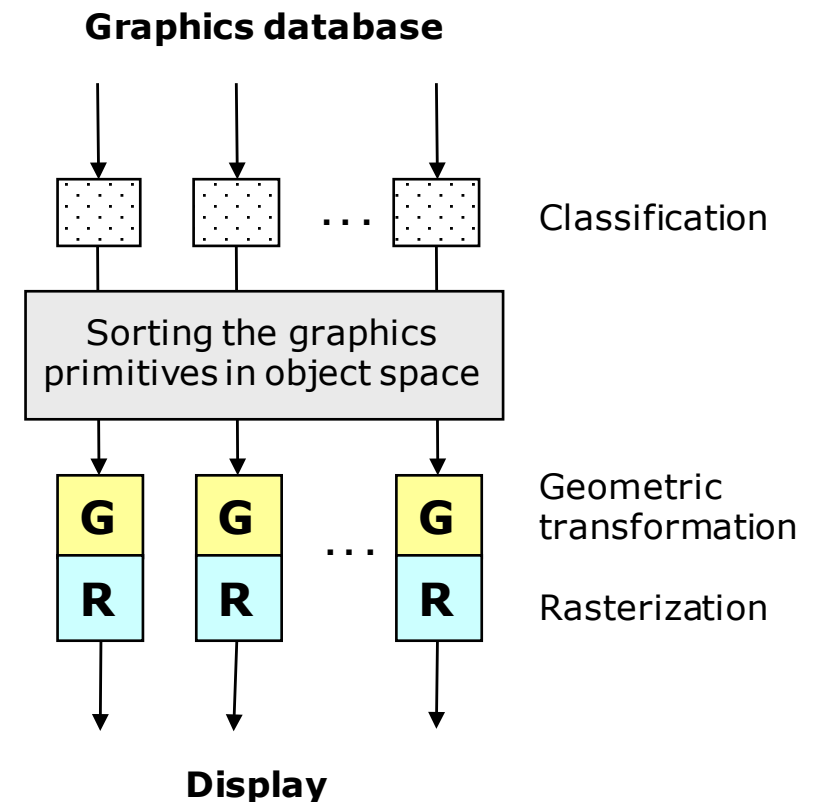
- divide input data into a collection of smaller components, each being processed by a computation node
- *sort-last* algorithms



Sort-first pipeline

□ Sort-first

- each processor is assigned a portion of the screen to render.
- classify the primitives according to their positions on the screen.
- during classification, the processors redistribute the primitives such that they all receive all of the primitives that fall in their respective portions of the screen.
- each processor performs the remaining transformation and rasterization steps for all of its resulting primitives.
- finished pixels are sent to one or more frame buffers to be displayed.

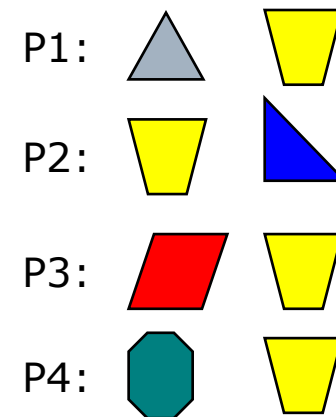
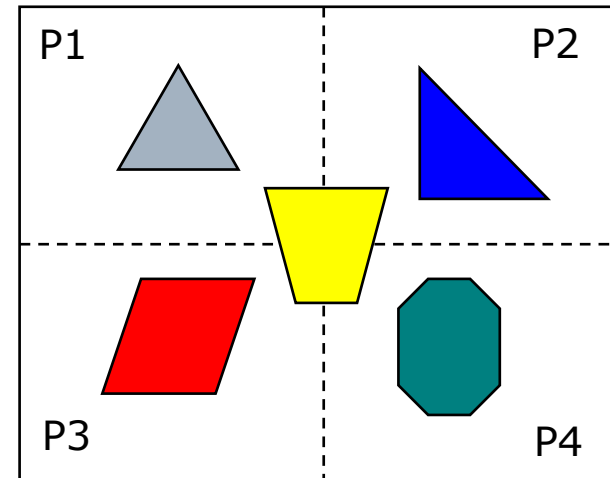


Sort first

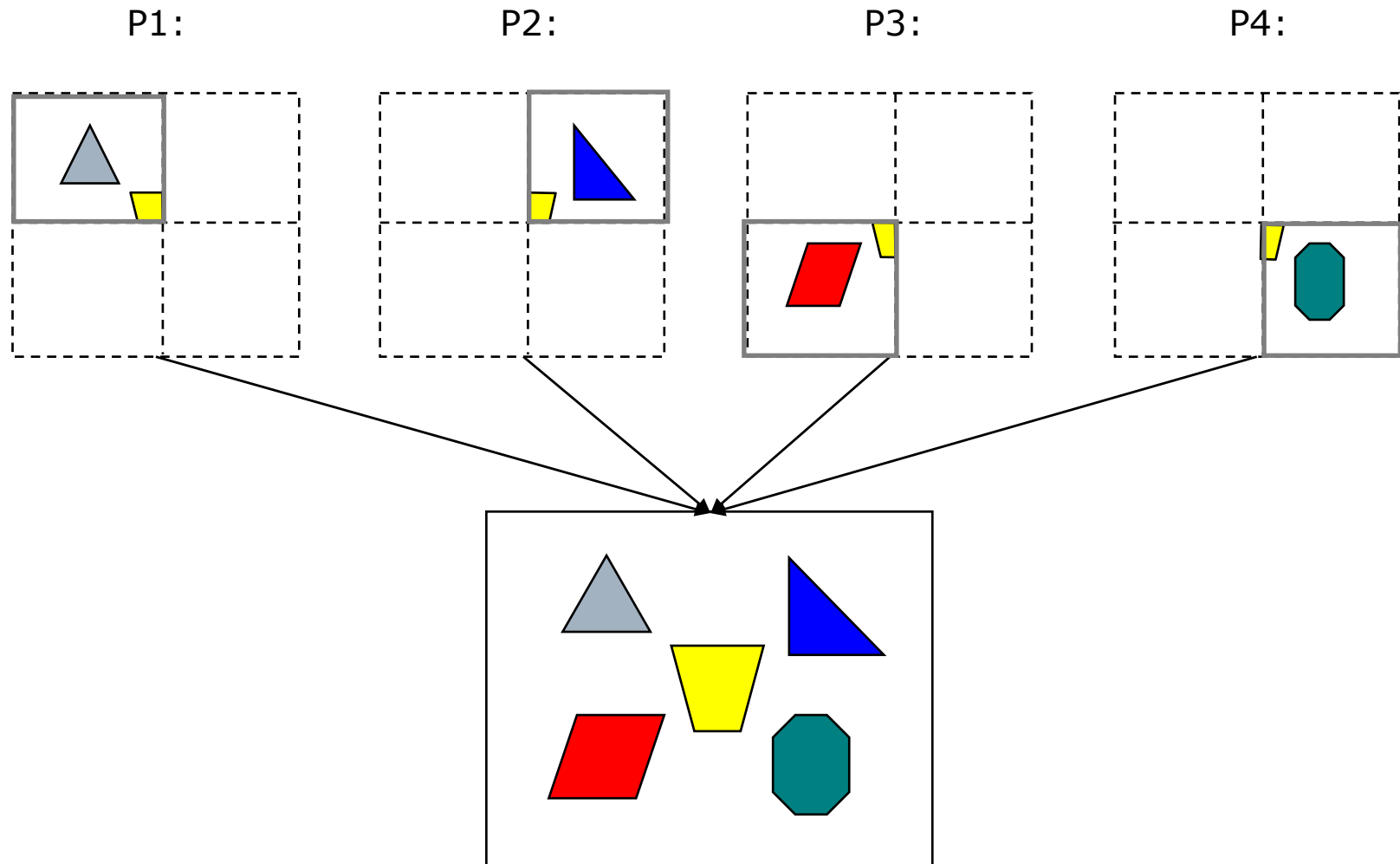
Distribute the primitives early in the rendering pipeline

Algorithm:

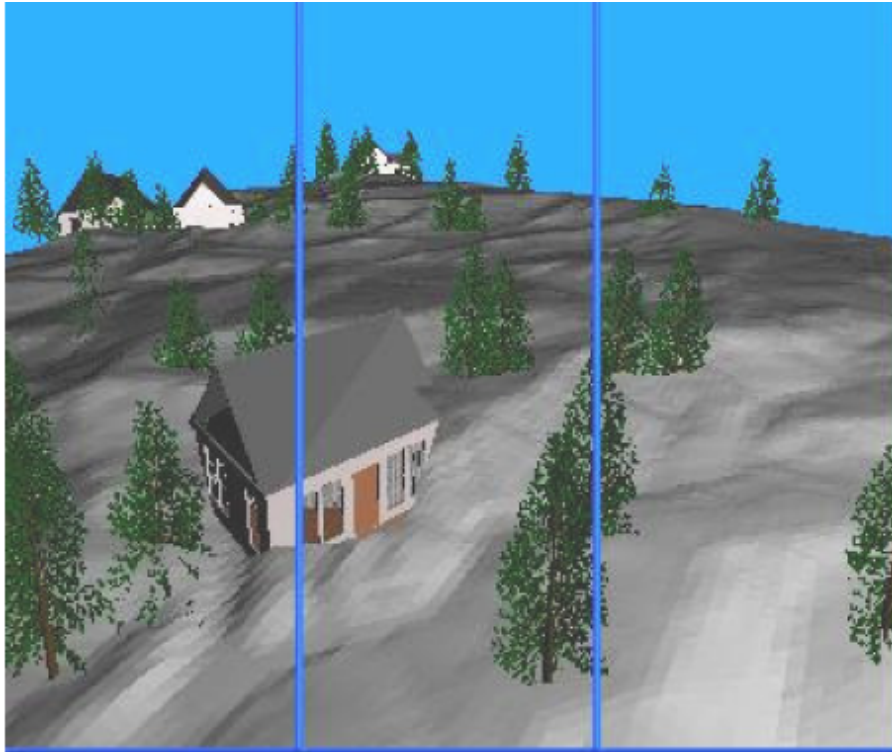
1. Subdivide the screen
2. Transform the primitives
3. Distribute the primitives
4. Each processor renders its own primitives
5. Does not require communication



Sort first



Sort first visualization

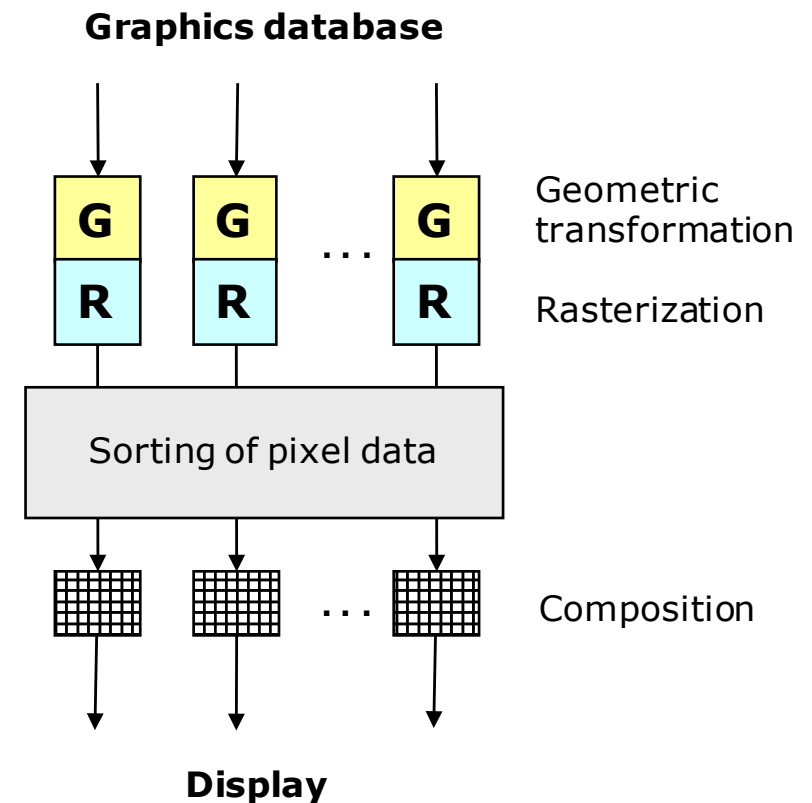


Bacu V., Muresan L., Gorgan D., Cluster Based Modeling and Visualization of Virtual Geographical Space. Workshop on Grid Computing Applications Development (GridCAD). Proceedings of the SYNASC 2008, IEEE Computer Press, pp. 416-421 (2008).

Sort-last pipeline

□ Sort-last

- each processor has a complete rendering pipeline and produces an incomplete full-area image by transforming and rasterizing its fraction of the primitives.
- these partial images are composited together, typically by depth sorting each pixel, in order to yield a complete image for the frame buffer.
- the composition step requires that pixel information (at least color and depth values) from each processor be sent across a network and sorted along the way to the frame buffer.

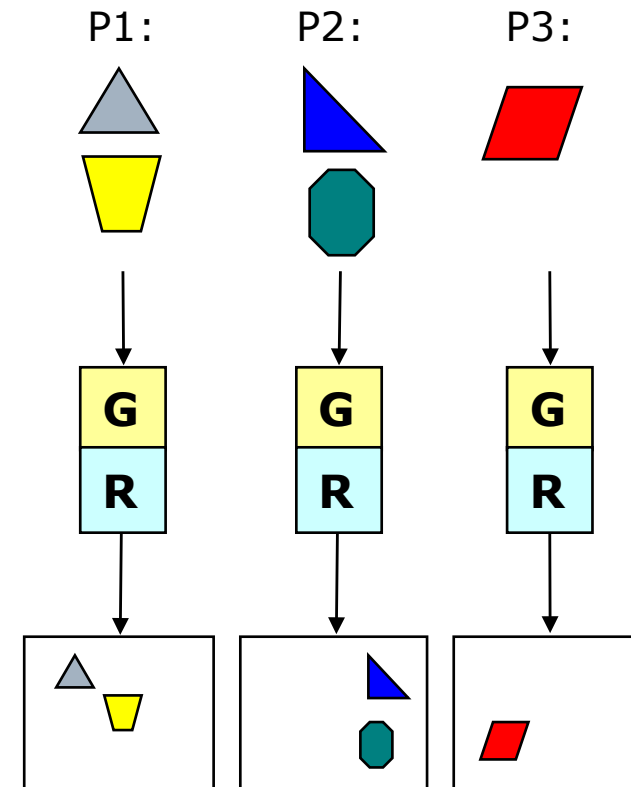
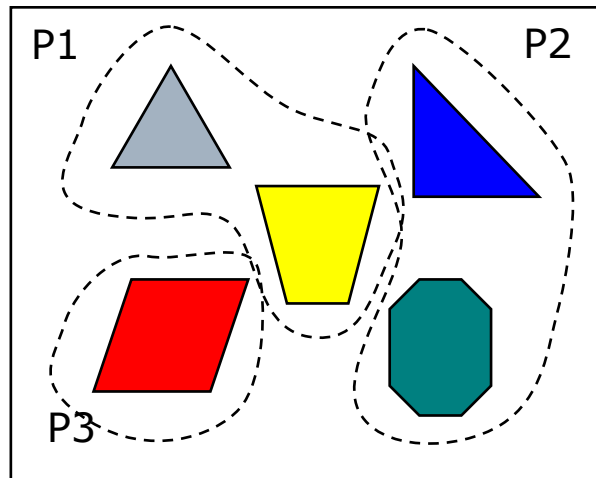


Sort last

Sort the primitives in the end of the pipeline

Algorithm:

1. Arbitrary assignment
2. Geometry processing
3. Rasterization

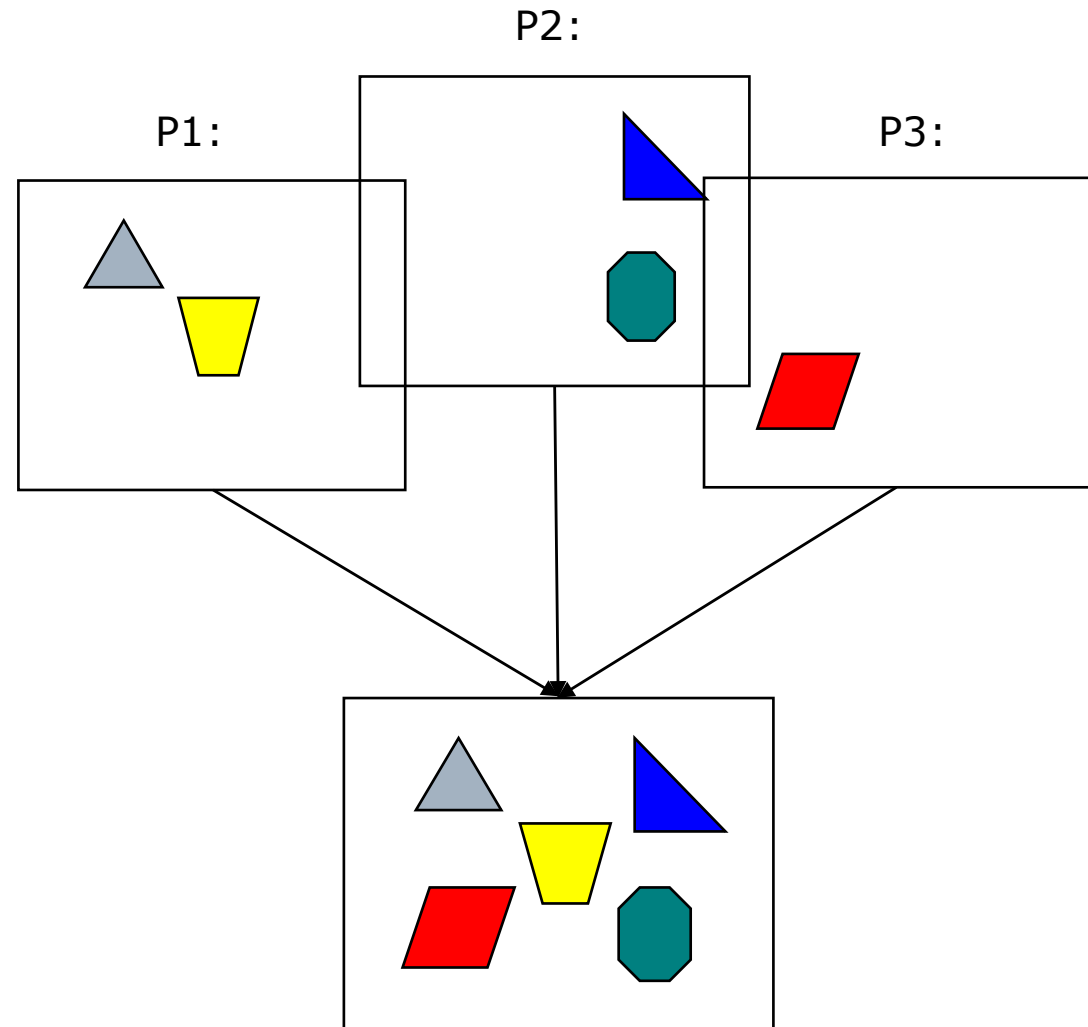
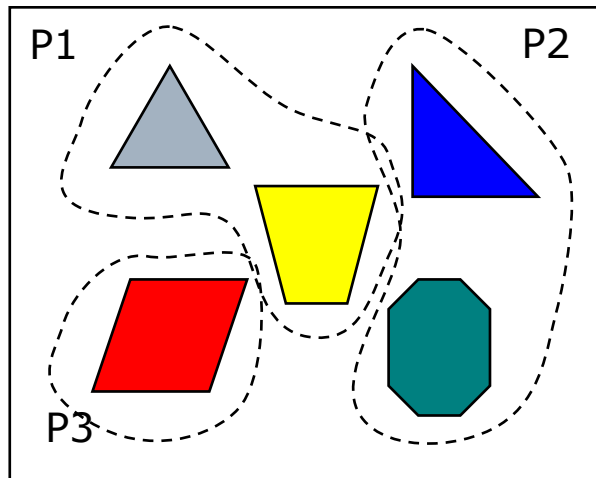


Sort last

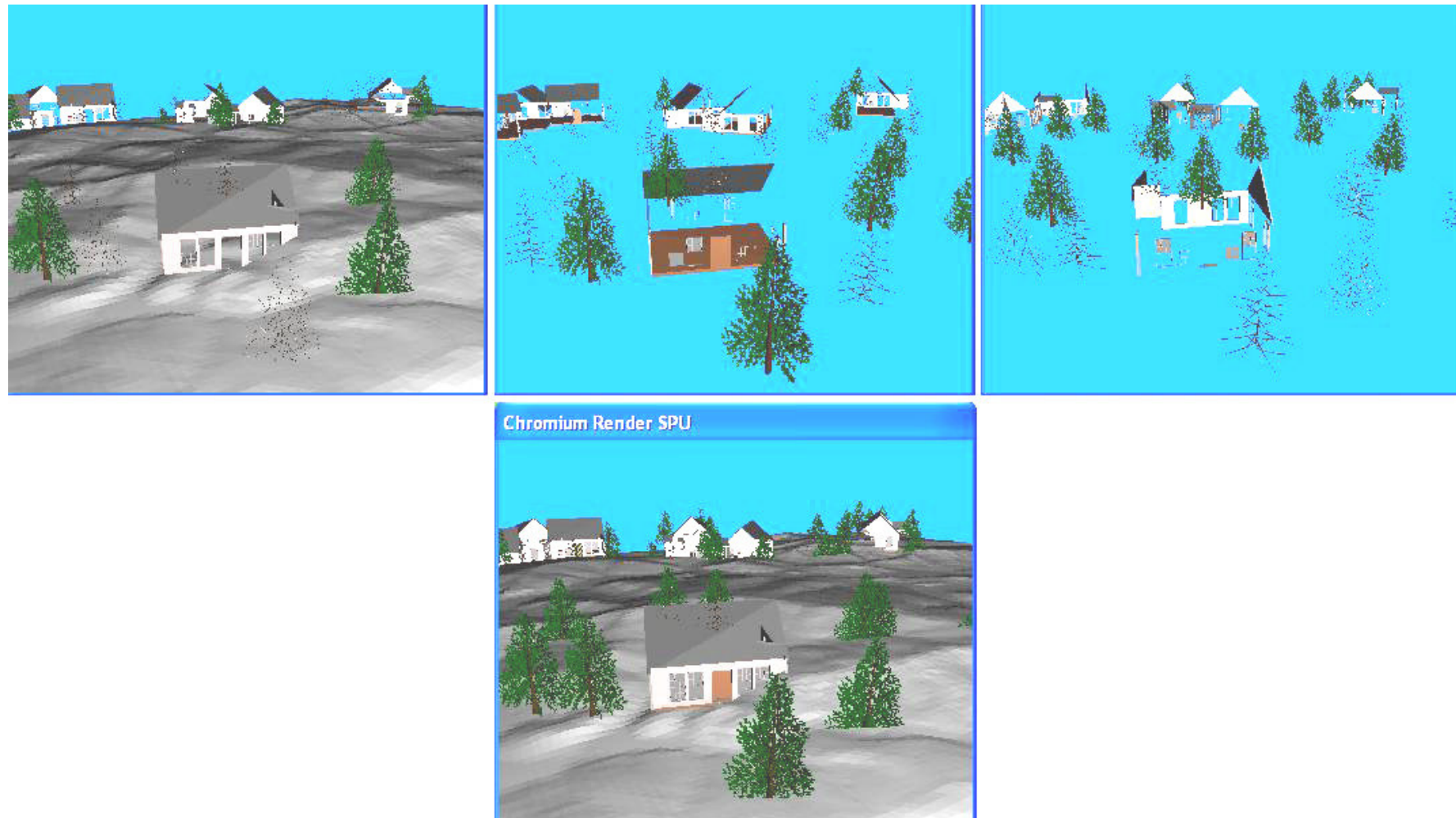
Sort the primitives in the end of the pipeline

Algorithm (cont):

4. Sort images by z
5. Composition/z-buffer



Sort-last visualization

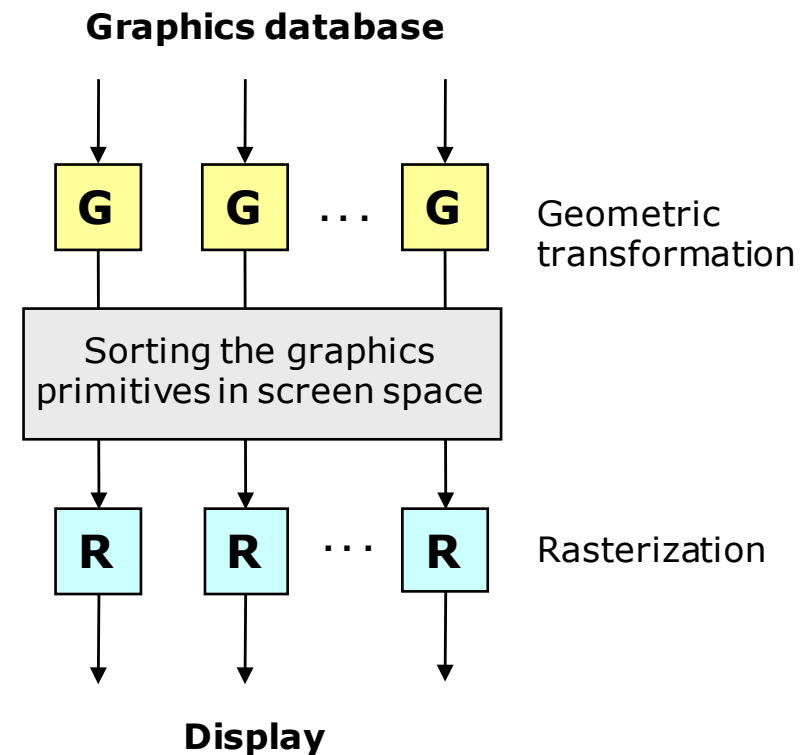


Bacu V., Muresan L., Gorgan D., Cluster Based Modeling and Visualization of Virtual Geographical Space. Workshop on Grid Computing Applications Development (GridCAD). Proceedings of the SYNASC 2008, IEEE Computer Press, pp. 416-421 (2008).

Sort-middle pipeline

□ Sort-middle

- there is a set of transformation processors and a set of rasterization processors.
- each rasterization processor is assigned a portion of the screen.
- to produce an image, each transformation processor completely transforms its portion of the primitives.
- the resulting primitive information is again classified by screen location and sent to the correct set of rasterization processors.
- after rasterization, finished pixels go to the frame buffer(s).



Conclusions

□ **Sort-first**

■ Frame-to-frame coherence

It takes advantage of frame-to-frame coherence. In an interactive application, the viewpoint changes very little from frame to frame, and thus the on-screen distribution of primitives does not change appreciably.

■ Reduced processor communication

Since primitives in a sort-first system are only transferred when they cross from one processor's screen region to another's, only a fraction of them will have to be communicated each frame. communication that does occur is typically fairly local; usually only "neighboring" processors will need to talk with each other.

■ Good scalability

It has good scalability in terms of the number of primitives it can handle.

■ Flexible rendering

in sort-first, once a processor has the correct set of primitives to render, only that processor is responsible for computing the final image for its portion of the screen. This allows great flexibility in terms of the rendering algorithms which may be used.

■ Very high resolution

since only finished pixels need to be sent to the frame-buffer, sort-first can easily handle very-high-resolution displays

■ Loadbalancing

Problem: loadbalancing. Because the on-screen distribution of primitives may be highly variable, also, managing a set of migrating primitives is a complex task

Conclusions

□ **Sort last**

- **Good scalability**

It offers excellent scalability in terms of the number of primitives it can handle.

- **Limited resolution**

Its pixel budget is limited by the bandwidth available at the composition stage.

- **Anti-aliasing**

It is a major problem for sort-last

- **Limited rendering algorithms**

Since visibility is not decided until after the composition stage, sort-last places limitations on the kinds of rendering algorithms which may be used. E.g. rendering transparent polygons

□ **Sort middle**

- **Limited scalability**

Any transformation processor must be able to talk to any rasterization processor → its scalability is limited.

- **Communications network**

Increasing the number of processors geometrically increases the demands on the communications network between them.

- **Load-balancing**

Load-balancing problems when the on-screen distribution of primitives is uneven. This will result in rasterization processors becoming unevenly loaded, and this in turn may degrade system performance

Load Balancing

- Strategy for mapping screen regions to processors
 - A simple strategy is to divide the screen into as many equal rectangles as there are processors and assigning them one-to-one can result in severe load-balancing problems. If the greatest concentration of primitives happens to fall into a single region, the parallel advantage of the system will be lost as the non-busy processors wait for the overloaded one to do its job.

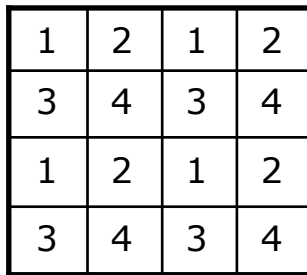


Fig 1. Static region assignment.

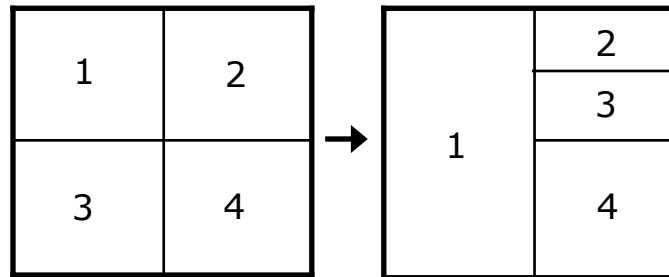


Fig 2. Roble's adaptive method. Regions 1 and 3 are combined. Processor 3 helps with the original region 2.

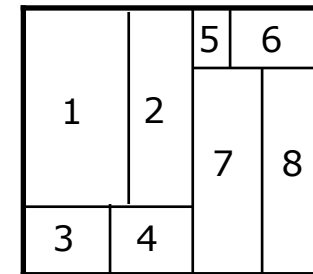


Fig 3. Whelan's adaptive method. The screen is recursively subdivided according to primitive centroids.

References

1. Burdea G.C., Coiffet P.: "Virtual Reality Technology", J.Wiley & Sons, Second Ed., 2003.
2. CS445: Intro Graphics, greg Humphreys, University of Virginia, Fall 2004.
3. Carl Mueller, The Sort-First Rendering Architecture for High-Performance Graphics. Department of Computer Science, University of North Carolina at Chapel Hill
4. R. Anderson, M. Carlsen, J. Postman, S. Rollins, GPU Technology, ECE 477 Spring 2007.
5. NVIDIA GeForce 8800 GPU Architecture Overview, November 2006 TB-02787-001_v01, <http://www.nvidia.com/page/geforce8.html>
6. GPUs, in ACM Queue, vol6, No.2, Murch/April 2008, <http://mags.acm.org/queue/20080304/>
7. Bacu V., Muresan L., Gorgan D., Cluster Based Modeling and Visualization of Virtual Geographical Space. Workshop on Grid Computing Applications Development (GridCAD). Proceedings of the SYNASC 2008, IEEE Computer Press, pp. 416-421 (2008).
8. Sabou, A., Gorgan, D., Parallel implicit time integration for particle-based models on graphics clusters. In Information and Communication Technology, Electronics and Microelectronics (MIPRO), IEEE 37th International Convention on, pp. 336-341 (2014).

Questions and problems

1. Describe the motivation of developing the high computation units such as GPU.
2. Identify in the historic evolution of GPU the gradually movement of graphics pipeline functionality into the integrated GPU.
3. What means GFLOPS, and why it could be considered a referenced capacity?
4. What are the main differences between CPU and GPU that give the high performance measured by thousands of GFLOPS? Argue by examples.
5. Why a GPU card module is much bigger than the integrated circuit itself (e.g. NVIDIA GeForce 8800 GTX)?

Questions and problems

6. What is a virtual GPU? Explain its usefulness.
7. What are the differences between the classical graphics pipeline and the GPU pipeline?
8. What is the main difference between vertex processing and triangle processing?
9. How is rasterized a triangle? Explain the computation of the fragment attributes.
10. What is the difference between a fragment and a pixel?
11. Explain the evolution of an OpenGL application and graphics model along the canonical graphics pipeline.
12. Exemplify the operations performed through a parallel graphics pipeline during the application, geometry and rasterizer stages.

Questions and problems

13. Explain what module gives the lowest processing speed along a graphics pipeline. How do you flow the processing throughout the graphics pipeline?
14. Exemplify the optimization of the processing flow in the application stage.
15. Exemplify the optimization of the processing flow in the geometry stage.
16. Exemplify the optimization of the processing flow in the rasterization stage.
17. Explain the concept of the unified shader architecture.
18. What is the main difference between OpenCL and CUDA?
19. Describe some software technologies used by GPU.

Questions and problems

20. Why the graphics visualization have to be distributed across a computing cluster?
21. Explain and exemplify the sort-first algorithms. Why it works in image space?
22. Explain and exemplify the sort-last algorithms. Why it works in object space?
23. Compare the sort-first and the sort-last algorithms. What are their main advantages and disadvantages?
24. Give an example of combining in parallel or in sequence within the same visualization the sort-first and the sort-last approaches.
25. Exemplify and explain the efficient mapping (load-balanced) across five processors, by (a) sort-first, and (b) sort-last methods of the following scene of objects:

Questions and problems

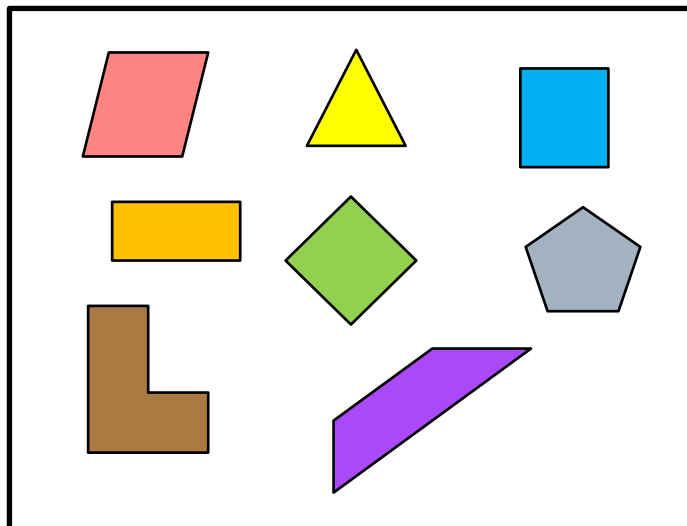


Fig. 1

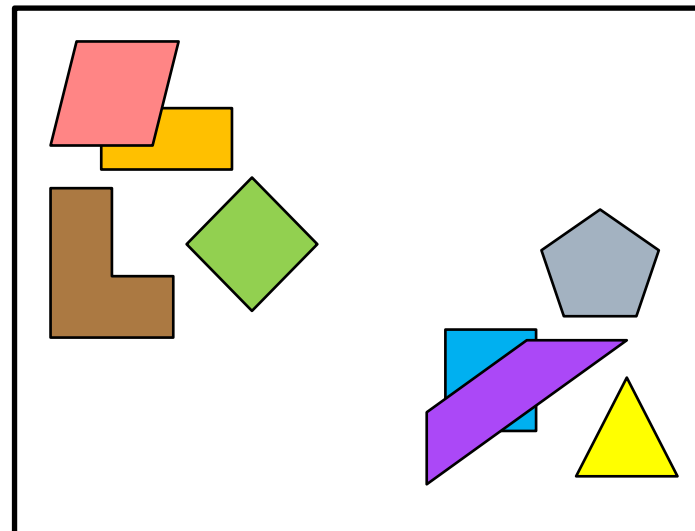


Fig. 2

Fig. 3

